FIELD MANUAL — SHELL DISCIPLINE

BASH

Patterns, Footguns & Combo Spells for Production Shell

A dense, load-bearing reference for writing Bash that survives contact with production: strict-mode reality, quoting discipline, safe filesystem operations, concurrency, and the footguns that separate scripts that work from scripts that only *appear* to work. No filler. No toy examples.

TARGET SHELL	COVERAGE	PORTABILITY NOTES	REVISION
Bash 4.x / 5.x	22 sections + appendix	GNU / BSD / macOS / Zsh	2026.07 — REV.C

§ 00

LEGEND & NAVIGATION

MARGIN MARKS | CONTENT MAP | READ THIS FIRST

This manual uses a fixed set of margin marks instead of prose warnings. Learn these six and the rest of the document reads at a glance — the mark tells you how much attention a block deserves before it tells you anything else.

`>_` **TIP / PATTERN**

A preferred idiom. Floated in the margin next to the code it improves on. Adopt by default.

▲ **WARNING**

Behavior that is correct but easy to misuse. Full-width banner — read before copying the block above it.

✗ **FOOTGUN**

Behavior that looks correct and silently isn't. These bite in code review, not at the terminal. Never skip these.

bash **BASH-SPECIFIC**

Requires Bash; will not run under POSIX `sh`, dash, or plain Bourne shell.

gnu **GNU-SPECIFIC**

Assumes GNU coreutils (Linux default). Flag or behavior differs or is absent on BSD/macOS userland.

bsd **BSD/macOS-SPECIFIC**

Assumes BSD userland (macOS default, no Homebrew coreutils). Differs from GNU.

00.2 CONTENT MAP – WHAT EACH SECTION TEACHES

§ 01
Command Pattern Index
One-page lookup table: task → idiom → section.

§ 02
Script Skeletons
Seven copy-paste starting points, from one-file utility to CI guard.

§ 03
Strict Mode Without Cargo Culting
What -e/-u/-o pipefail actually do, and where they lie to you.

§ 04
Quoting & Expansion Footguns
Word splitting, globbing, and the eval trapdoor. The core danger zone.

§ 05
Variables, Parameters & Defaults
${var:-default} family, namerefs, indirect expansion.

§ 06
Arrays & Maps
Indexed/associative arrays as the antidote to string-built commands.

§ 07

Argument Parsing

Manual while/case parsers, subcommands, -- sentinel, getopts limits.

§ 08

Functions & Return Behavior

stdout-for-data / stderr-for-logs discipline, namerefs, libraries.

§ 09

Filesystem-Safe Scripting

NUL-delimited pipelines, atomic writes, safe deletion, backups.

§ 10

Pipelines, Subshells & Process Substitution

Where variables silently die, and how to keep them alive.

§ 11

Text Processing Patterns

When to reach for awk, sed, jq, or Python instead of Bash itself.

§ 12

External Commands & Dependencies

GNU vs. BSD tool detection, version guards, command -v.

§ 13

Logging, Verbosity & UX

Log levels, TTY-aware color, --json output, usage text.

§ 14

Traps, Cleanup & Signals

EXIT/INT/TERM/ERR traps, exit-status preservation, child reaping.

§ 15

Concurrency & Batching

Background jobs, wait -n, xargs -P, GNU parallel, failure collection.

§ 16

Networking & API Scripting

curl retries/timeouts, jq -n payloads, pagination, token hygiene.

§ 17

Security Hardening

Injection defense, PATH hygiene, secrets, umask, ShellCheck gates.

§ 18

Debugging & Inspection

bash -x, PS4 tracing, BASH_XTRACEFD, declare -p, minimal repros.

§ 19

Testing Bash Scripts

Smoke tests, Bats-core, PATH mocking, dry-run destructive paths.

§ 20

Portability Matrix

Bash 3.2/4/5, Zsh, POSIX sh, GNU vs. BSD userland — compact grid.

§ 21

Advanced Combo Patterns

Eleven full field-tested spells: audits, fetchers, parallel batches.

§ 22

Appendix

Reference tables: test operators, expansions, exit codes, ShellCheck codes.

§ 01

COMMAND PATTERN INDEX

LOOKUP TABLE | ONE PAGE | TASK → IDIOM

Grouped by task. Find the row, jump to the section for the full pattern, footguns, and portability notes.

TASK	CANONICAL IDIOM	§
Argument parsing	`while (($#)); do case "$1" in`	§07
Strict mode	`set -Eeuo pipefail; trap 'die "line $LINENO"' ERR`	§03
Arrays	`local -a args=(); args+=("$x")`	§06
String manipulation	`${var##*/} ${var%.*} ${var//foo/bar}`	§05
File loops	`while IFS= read -r -d '' f; do ... done < <(find . -print0)`	§09
Safe deletion	`confirm_and_rm() { [[ -n $1 ]] || die "empty path"; }`	§09
Temp files	`tmp=$(mktemp) && trap 'rm -f "$tmp"' EXIT`	§09/14
Traps	`trap cleanup EXIT INT TERM; trap 'on_err $LINENO' ERR`	§14
Logging	`log() { printf '%s [%s] %s\n' "$(date -u +%FT%TZ)" "$1" "$2" >&2; }`	§13
Pipelines	`set -o pipefail; cmd1 | cmd2 | cmd3`	§10
Process substitution	`diff <(sort a) <(sort b)`	§10
Command substitution	`out="$(cmd)" ; status=$?`	§10
Quoting	`"${arr[@]}" not ${arr[@]} ; "$var" not $var`	§04
Globbing	`shopt -s nullglob dotglob; for f in *.log; do`	§04
Regex matching	`[[ $s =~ ^([0-9]+)-([a-z]+)$ ]] && echo "${BASH_REMATCH[1]}"`	§11
Config files	`while IFS='=' read -r k v; do declare "$k=$v"; done < cfg`	§11
Concurrency	`cmd & pids+=($!); wait -n; wait "${pids[@]}"`	§15
curl / API scripts	`curl --fail-with-body --retry 3 -sS -H "Authorization: Bearer $TOK"`	§16
JSON with jq	`jq -r '.items[] | [.id,.name] | @tsv'`	§16/11
Security hardening	`umask 077; readonly -a SAFE_PATH=(/usr/bin /bin)`	§17
Debugging	`PS4='+ ${BASH_SOURCE}:${LINENO}:${FUNCNAME[0]:-main} ' bash -x script.sh`	§18
Portability footguns	`readlink -f is GNU-only; use physical cd fallback on macOS`	§20

READ ORDER

This index is a map, not a tutorial sequence. Sections are self-contained — jump directly to the one you need. §03 and §04 (strict mode, quoting) are the two most load-bearing sections if you only have time for two.

§ 02

SCRIPT SKELETONS

COPY-PASTE | VALIDATED WITH SHELLCHECK | NO FAKE TODOS

Seven starting points. Every skeleton passes `shellcheck -x` clean. Common spine across all of them: explicit `IFS`, a `die()` or leveled logger, a trap-driven cleanup path, and exit codes chosen on purpose — not whatever bubbled up.

IFS=$'\N\T'

Strips the default space from word-splitting so unquoted expansions in loops don't fracture on internal spaces. Doesn't fix everything — you still need `"$var"` quoting. See §04.

02.1 ONE-FILE UTILITY

SKEL_UTILITY.SH

```
#!/usr/bin/env bash
# skel_utility.sh — small one-file utility
set -Eeuo pipefail
IFS=$'\n\t'

readonly PROG="${BASH_SOURCE[0]##*/}"

usage() { printf 'usage: %s <input-file>\n' "$PROG" >&2; }
die()   { printf '%s: error: %s\n' "$PROG" "$*" >&2; exit 1; }

main() {
  (( $# == 1 )) || { usage; exit 2; }
  local input=$1
  [[ -r "$input" ]] || die "cannot read: $input"
  awk '{ sum += $1 } END { printf "%.2f\n", sum }' "$input"
}

main "$@"
```

02.2 AUDIT / REPORT SCRIPT

EXIT CODE AS SIGNAL

Returning non-zero when findings exist (not just on crash) lets CI gate on the audit directly — no separate parsing step.

SKEL_AUDIT.SH

```bash
#!/usr/bin/env bash
# skel_audit.sh — audit/report script: read-only checks, structured findings
set -Eeuo pipefail
IFS=$'\n\t'

readonly PROG="${BASH_SOURCE[0]##*/}"
FINDINGS=()

log()  { printf '%s [%s] %s\n' "$PROG" "$(date -u +%H:%M:%S)" "$*" >&2; }
finding() { FINDINGS+=("$1|$2"); }   # severity|message

check_world_writable() {
  local f
  while IFS= read -r -d '' f; do
    finding "HIGH" "world-writable: $f"
  done < <(find /etc -maxdepth 2 -perm -0002 -type f -print0 2>/dev/null)
}

report() {
  (( ${#FINDINGS[@]} )) || { log "clean"; return 0; }
  printf '%-6s %s\n' "SEV" "FINDING"
  local entry sev msg
  for entry in "${FINDINGS[@]}"; do
    IFS='|' read -r sev msg <<< "$entry"
    printf '%-6s %s\n' "$sev" "$msg"
  done
}

main() {
  log "starting audit"
  check_world_writable
  report
  (( ${#FINDINGS[@]} == 0 ))   # exit 1 if findings exist (CI-friendly)
}

main "$@"
```

02.3 SUBCOMMAND DISPATCH

SKEL_SUBCOMMANDS.SH

```
#!/usr/bin/env bash
# skel_subcommands.sh — dispatch to subcommand functions
set -Eeuo pipefail
IFS=$'\n\t'

readonly PROG="${BASH_SOURCE[0]##*/}"
die() { printf '%s: %s\n' "$PROG" "$*" >&2; exit 1; }

usage() {
  cat >&2 <<EOF
usage: $PROG <command> [args]
commands:
  status         show current state
  sync <target>  sync to target
EOF
}

cmd_status() { printf 'status: ok\n'; }
cmd_sync()   { local target="${1:?sync requires <target>}"; printf 'syncing to %s\n' "$target"; }

main() {
  (( $# )) || { usage; exit 2; }
  local cmd=$1; shift
  case "$cmd" in
    status) cmd_status "$@" ;;
    sync)   cmd_sync "$@" ;;
    -h|--help) usage; exit 0 ;;
    *) die "unknown command: $cmd" ;;
  esac
}

main "$@"
```

§ 02

SCRIPT SKELETONS (CONT.)

CONFIG | LOGGING | CI | BULK FILES

02.4 CONFIG FILE + ENVIRONMENT VARIABLES

PRINTF -V INJECTION

Building a variable name from untrusted config keys (`printf -v "CFG_$key"`) is safe only because keys are filtered to a config file you control. Never do this with network-sourced keys — see §17.

SKEL_CONFIG.SH

```
#!/usr/bin/env bash
# skel_config.sh — config file + environment variable precedence
set -Eeuo pipefail
IFS=$'\n\t'

readonly PROG="${BASH_SOURCE[0]##*/}"
readonly CONFIG_DEFAULT="${XDG_CONFIG_HOME:-$HOME/.config}/${PROG%.sh}/config"

: "${APP_ENV:=production}"
: "${APP_CONFIG:=$CONFIG_DEFAULT}"
: "${APP_TIMEOUT:=30}"

die() { printf '%s: %s\n' "$PROG" "$*" >&2; exit 1; }

load_config() {
  [[ -f "$APP_CONFIG" ]] || return 0
  local key val
  while IFS='=' read -r key val; do
    [[ $key =~ ^[[:space:]]*# ]] && continue
    [[ -z $key ]] && continue
    key=${key// /}
    printf -v "CFG_${key}" '%s' "$val"
  done < "$APP_CONFIG"
}

main() {
  load_config
  [[ $APP_ENV =~ ^(production|staging|dev)$ ]] || die "invalid APP_ENV: $APP_ENV"
  printf 'env=%s timeout=%s config=%s\n' "$APP_ENV" "$APP_TIMEOUT" "$APP_CONFIG"
}

main "$@"
```

02.5 LOGGING + GUARANTEED CLEANUP

SKEL_LOGGING_CLEANUP.SH

```bash
#!/usr/bin/env bash
# skel_logging_cleanup.sh — leveled logging + guaranteed cleanup
set -Eeuo pipefail
IFS=$'\n\t'

readonly PROG="${BASH_SOURCE[0]##*/}"
WORKDIR=""

log_info()  { printf '%s: [INFO ] %s\n' "$PROG" "$*" >&2; }
log_error() { printf '%s: [ERROR] %s\n' "$PROG" "$*" >&2; }
die()       { log_error "$*"; exit 1; }

cleanup() {
  local rc=$?
  [[ -n "$WORKDIR" && -d "$WORKDIR" ]] && rm -rf -- "$WORKDIR"
  (( rc != 0 )) && log_error "exiting with status $rc"
  exit "$rc"
}
trap cleanup EXIT
trap 'die "failed at line $LINENO: $BASH_COMMAND"' ERR

main() {
  WORKDIR=$(mktemp -d) || die "mktemp -d failed"
  log_info "workdir: $WORKDIR"
  : > "$WORKDIR/marker"
  log_info "done"
}

main "$@"
```

02.6 SCRIPT INTENDED FOR CI

::GROUP:: SYNTAX

The `::group::` / `::endgroup::` convention collapses output in GitHub Actions logs. Swap for your CI's own log-folding syntax; the exit-code discipline underneath is what matters.

SKEL_CI.SH

```
#!/usr/bin/env bash
# skel_ci.sh — script intended for CI: deterministic exit codes, no interactivity
set -Eeuo pipefail
IFS=$'\n\t'

readonly EXIT_OK=0 EXIT_LINT=10 EXIT_TEST=11 EXIT_BUILD=12
# NO_COLOR is a convention read by downstream tools (see no-color.org); not used directly here.
# shellcheck disable=SC2034
[[ -t 1 ]] || NO_COLOR=1
: "${CI:=false}"

log() { printf '::%s:: %s\n' "$1" "$2"; }

run_step() {
  local name=$1; shift
  log group "$name"
  if "$@"; then
    log endgroup "$name ok"
  else
    local rc=$?
    log endgroup "$name FAILED ($rc)"
    return "$rc"
  fi
}

main() {
  run_step lint    shellcheck -x ./bin/*.sh          || exit "$EXIT_LINT"
  run_step test    bats test/                        || exit "$EXIT_TEST"
  run_step build   docker build -t app:ci .           || exit "$EXIT_BUILD"
  exit "$EXIT_OK"
}

main "$@"
```

02.7 PROCESS MANY FILES SAFELY

SKEL_MANY_FILES.SH

```bash
#!/usr/bin/env bash
# skel_many_files.sh — process many files safely (spaces, newlines, globs)
set -Eeuo pipefail
IFS=$'\n\t'
shopt -s nullglob

readonly PROG="${BASH_SOURCE[0]##*/}"
DRY_RUN=0

die() { printf '%s: %s\n' "$PROG" "$*" >&2; exit 1; }

process_one() {
  local f=$1
  [[ -f "$f" ]] || return 0
  if (( DRY_RUN )); then
    printf 'would process: %s\n' "$f"
  else
    gzip -k -- "$f"
  fi
}

main() {
  local root=${1:?usage: PROG <root-dir> [--dry-run]}
  [[ ${2:-} == --dry-run ]] && DRY_RUN=1
  [[ -d "$root" ]] || die "not a directory: $root"

  local f
  while IFS= read -r -d '' f; do
    process_one "$f"
  done < <(find "$root" -type f -name '*.log' -print0)
}

main "$@"
```

FOOTGUN

None of these skeletons trap on a bare `set -e` alone. Pipelines, conditionals, and command substitutions all interact with `-e` in ways that silently swallow failures. Full treatment in §03.

§ 03

STRICT MODE WITHOUT CARGO CULTING

SET -EEUO PIPEFAIL | ERR TRAPS | EDGE CASES

The four flags do less than the copy-pasted header implies. Know exactly which constructs they cover, or the header is decorative.

03.1 WHAT EACH FLAG ACTUALLY DOES

FLAG	EFFECT	DOES NOT COVER
`set -e`	Exit if a simple command returns non-zero.	Commands in `if` / `while` conditions, before `&&` / `||`, or the non-last stage of a pipeline.
`set -u`	Error on unset variable expansion.	Array subscripts on unset arrays under some Bash 4.3 versions; `$*` in some edge builds — verify on target Bash.
`set -o pipefail`	Pipeline exit status = rightmost non-zero stage, not just the last.	Does not by itself trigger `-e` inside a pipeline used as a condition.
`set -E`	ERR trap inherited by functions and subshells.	Without `-E`, a `trap ... ERR` silently fails to fire inside functions.

03.2 SET -E EDGE CASES — BROKEN EXPECTATIONS

CONDITIONS DISARM -E

Any command used as a condition (`if` , `while` , `&&` , `||` , negated with `!`) does not trigger `-e` even if it fails deep inside a function call chain.

BROKEN-EXPECTATION.SH — WON'T EXIT

```
set -e
check_disk() { df -h | grep -q "100%"; }   # returns 1 if no match
if check_disk; then
  echo "disk full"
fi
echo "script continues normally either way"  # ALWAYS reached
```

NON-LAST PIPELINE STAGE

Without `pipefail` , only the exit status of the final command in a pipeline is visible to `-e` . An early failing producer is invisible.

MISSING-PIPEFAIL.SH

```
set -e   # no pipefail
grep pattern /nonexistent/file | sort | uniq -c
echo "reached even though grep failed"
```

WITH-PIPEFAIL.SH FIXED

```
set -eo pipefail
grep pattern /nonexistent/file | sort | uniq -c
echo "not reached: grep's failure now propagates"
```

03.3 COMMAND SUBSTITUTION AND SUBSHELLS

COMMAND SUBSTITUTION

Failure inside `$(...)` does NOT stop the outer statement unless the substitution result is itself used somewhere that checks status — assignment alone doesn't check it.

CMDSUB-MASKING.SH

```
set -e
out="$(false; echo "still ran")"
echo "$out"          # prints: still ran
echo "assignment RHS masked the failing 'false'"
```

03.4 EXPLICIT CHECKS BEAT IMPLICIT TRUST

EXPLICIT-CHECKS.SH

```
die() { printf 'fatal: %s\n' "$*" >&2; exit 1; }

# Preferred: explicit, greppable, works inside conditions
cp -- "$src" "$dst" || die "copy failed: $src -> $dst"

if ! curl -fsS "$url" -o "$out"; then
  die "download failed: $url"
fi

# Preserve the real exit status, don't collapse it to 1
run_job || { rc=$?; log_error "job failed rc=$rc"; exit "$rc"; }
```

03.5 ERR TRAP: THE -E SAFETY NET

ERR-TRAP.SH

```
set -Eeuo pipefail
on_error() {
  local rc=$? line=$1
  printf 'ERROR: line %s exited %s: %s\n' "$line" "$rc" "$BASH_COMMAND" >&2
  exit "$rc"
}
trap 'on_error $LINENO' ERR
```

REQUIRES SET -E

Without `-E`, this trap fires at top level but is silently skipped inside functions and command substitutions — exactly where you need it most.

03.6 WHEN NOT TO USE STRICT MODE

- Interactive-facing scripts that should degrade gracefully instead of hard-exiting on a single missing optional tool.
- Scripts that intentionally probe for failure (`command -v` chains, capability detection) — wrap those probes in `if` so `-e` doesn't fight the probe.
- Sourced library files: setting `-e` in a sourced file changes the caller's shell options. Prefer per-function `|| return 1` discipline instead.

FOOTGUN

A function called as the last statement in another function inherits that function's return status. If cleanup logic runs after via a trap and itself succeeds, it can silently overwrite a real failure's exit code. Capture `rc=$?` as the very first statement in any EXIT trap.

§ 04

QUOTING & EXPANSION FOOTGUNS

THE DANGER ZONE | WORD SPLITTING | GLOBBING | EVAL

Nearly every "it worked on my machine" Bash incident traces back to one of the patterns on this page. Read it twice. Quote by default; treat every bare `$var` outside `[[ ]]` arithmetic context as a bug until proven otherwise.

04.1 "${VAR}" – QUOTE BY DEFAULT

BASIC-QUOTING.SH

```
name="Ada Lovelace"
printf 'hi %s\n' $name     # BAD: splits into "hi", "Ada" then "Lovelace" as extra printf args
printf 'hi %s\n' "$name"   # GOOD: one field, exactly as stored
```

04.2 ${ARRAY[@]} VS. ${ARRAY[*]}

IFS JOINS THE STAR

`"${arr[*]}"` joins every element with the first char of `IFS` into ONE string/ argument. `"${arr[@]}"` expands to N separate, individually-quoted words. Almost always want `@`.

ARRAY-EXPANSION.SH

```
args=("--flag" "value with spaces" "-x")
cmd "${args[@]}"   # GOOD: 3 distinct argv entries, spaces preserved
cmd "${args[*]}"   # BAD: 1 argv entry: "--flag value with spaces -x"
cmd ${args[@]}     # BAD: unquoted -> word-split -> 5 argv entries, spaces gone
```

04.3 UNQUOTED GLOBS & EMPTY VARIABLES

EMPTY-VAR-GLOB.SH — CLASSIC OUTAGE

```
rm -rf $dir/*        # if $dir is unset or empty: rm -rf /* candidate after globbing in cwd
rm -rf -- "${dir:?}"/*   # GOOD: aborts with an error if dir is unset or empty, -- stops flag injection
```

THE EMPTY-VARIABLE BLAST RADIUS

`${dir:?}` triggers an immediate error-and-exit if `dir` is unset or empty, before the glob ever expands. This single idiom has prevented more real incidents than any other line in this manual.

04.4 WORD SPLITTING & PATHNAME EXPANSION

WORD-SPLITTING.SH

```
files=$(ls *.txt)      # BAD: word-splits on IFS, re-globs each resulting word
for f in $files; do ...              # breaks on spaces, silently mis-parses filenames

shopt -s nullglob
files=(*.txt)                        # GOOD: real array, no subshell, no re-parsing
for f in "${files[@]}"; do ...
```

NULLGLOB

Without `shopt -s nullglob`, a glob that matches nothing expands to the literal pattern string itself (e.g. `*.txt`), which then gets treated as a filename.

04.5 COMMAND SUBSTITUTION STRIPS TRAILING NEWLINES

CMDSUB-NEWLINES.SH

```
printf 'a\nb\n\n\n' > f
x="$(cat f)"
printf '[%s]\n' "$x"    # prints [a\nb] -- ALL trailing newlines are gone, not just one
# If trailing newlines matter (binary-safe capture), append a sentinel and strip it:
x="$(cat f; printf X)"; x="${x%X}"
```

04.6 NESTED QUOTING

NESTED-QUOTING.SH

```
# Single quotes can't contain a single quote -- close, escape, reopen:
echo 'it'\''s broken'          # -> it's broken
# Cleaner for one embedded apostrophe: switch to double quotes for that segment
echo "it's fine"
# Deeply nested: build with printf %q or avoid nesting via a function/array instead of a string
```

04.7 ANSI-C QUOTING & PRINTF %Q

ANSI-C-AND-PRINTF-Q.SH

```
sep=$'\t'                # ANSI-C quoting: real tab, not literal backslash-t
nl=$'\n'
printf 'a%sb\n' "$sep"

safe=$(printf '%q' "$user_input")   # shell-escape for safe *re-use* as shell input
printf 'Run: %s\n' "$safe"          # display form is NOT automatically execution-safe -- see 04.8
```

04.8 SHELL-SAFE DISPLAY VS. SHELL-SAFE EXECUTION

PRINTF %Q IS NOT A SANDBOX

`%q` produces a string that, if fed back through the shell, reproduces the original value. It is for logging/debugging, not for building a command you then hand to `eval` or `ssh host "cmd"`. Use arrays (§06) for constructing commands, not string concatenation plus escaping.

04.9 QUOTING INSIDE AWK, SED, FIND, XARGS, SSH

TOOL-QUOTING.SH

```
# awk: pass shell values as -v assignments, never interpolate into the awk program text
awk -v pat="$needle" '$0 ~ pat { print }' file

# sed: same principle -- build the expression from -e with a quoted variable, watch sed's own metachars
sed -e "s/${needle//\//\\/}/${repl//\//\\/}/g" file

# find -exec: use the {} + form; it batches args and never re-splits them through a shell
find . -name '*.log' -exec gzip {} +

# xargs: -0 with -print0 upstream; -I{} is safe for spacey names, but forks once per item
find . -name '*.log' -print0 | xargs -0 -I{} gzip {}

# ssh: the remote command is re-parsed by the REMOTE shell -- quote once for local, once for remote
ssh host "gzip -- $(printf '%q' "$remote_path")"
```

FOOTGUN

Every one of the tools above re-parses its argument through its own grammar (awk expressions, sed expressions, a remote shell). Quoting correctly for Bash is necessary but not sufficient — you are quoting twice, for two different parsers, and they do not agree on metacharacters.

04.10 WHY EVAL IS USUALLY A TRAPDOOR

EVAL-TRAPDOOR.SH

```
cmd="ls -la /tmp"
eval "$cmd"                # fine here -- but the moment any part of $cmd is attacker- or user-
influenced:
filename='; rm -rf ~ #'
eval "ls $filename"        # constructs and runs: ls ; rm -rf ~ #
```

EVAL CONCATENATES TRUST LEVELS

`eval` re-parses its argument as a brand-new command line. Any untrusted substring anywhere in that argument gets shell metacharacter privileges. If you think you need `eval`, you almost always need an array or a nameref instead (§06, §08).

§ 05

VARIABLES, PARAMETERS & DEFAULTS

EXPANSION FAMILY | NAMEREFS | SCOPE

The `${var:-default}` family covers most default-handling without a single `if`. Learn the four variants by their punctuation, not by example lookup.

05.1 THE FOUR-SYMBOL FAMILY

FORM	MEANING
`${var:-default}`	Use *default* if unset/empty. **Does not assign** — var stays unset.
`${var:=default}`	Use and **assign** *default* if unset/empty. Fails on positional params / array elements.
`${var:?message}`	Print *message* to stderr and exit (non-interactive) if unset/empty.
`${var:+value}`	Use *value* only if var IS set/non-empty — inverse of `:-`.

> **DROP THE COLON FOR "SET BUT POSSIBLY EMPTY"**
>
> Omitting the `:` (e.g. `${var-default}`) checks only for *unset*, treating an explicitly empty string as already-set. Rarely what you want for paths.

05.2 SUBSTRING, PREFIX/SUFFIX, PATTERN REPLACEMENT

EXPANSION-OPS.SH

```
path="/var/log/app/current.log"
echo "${path:5}"          # substring from offset 5
echo "${path:5:8}"        # substring, offset 5, length 8
echo "${path#*/}"         # remove shortest matching prefix  -> var/log/app/current.log
echo "${path##*/}"        # remove longest matching prefix   -> current.log   (basename)
echo "${path%/*}"         # remove shortest matching suffix  -> /var/log/app  (dirname)
echo "${path%%.*}"        # remove longest matching suffix   -> /var/log/app/current
echo "${path/log/LOG}"    # replace first match
echo "${path//\//_}"      # replace all matches (here: every /)
```

05.3 INDIRECT EXPANSION & NAMEREFS

NAMEREFS.SH

```
# Indirect (legacy, string-based -- fragile, avoid in new code):
varname="HOME"
echo "${!varname}"

# Nameref (Bash 4.3+, preferred -- a real alias to another variable):
declare -n ref=SOME_VAR
SOME_VAR="hello"
echo "$ref"                # hello -- ref tracks SOME_VAR by reference
```

NAMEREF SELF-REFERENCE

If a nameref's target name collides with a local inside the same function (including the parameter name used to receive it), Bash raises a circular-reference error. Prefix nameref-receiving parameters distinctly, e.g. `_ref`.

05.4 READONLY, LOCAL, AND AVOIDING GLOBAL SOUP

SCOPE-DISCIPLINE.SH

```
readonly MAX_RETRIES=5          # constants: fail loudly if something tries to reassign

configure() {
  local -r timeout=30           # local AND readonly, scoped to this function only
  local endpoint="${1:?endpoint required}"
  printf 'timeout=%s endpoint=%s\n' "$timeout" "$endpoint"
}
```

LOCAL RETURN-STATUS GOTCHA

`local x=$(cmd)` discards the exit status of `cmd` -- `local` itself is the last command, so `$?` reflects the assignment, not `cmd`. Split into two statements if you need the status.

05.5 EXPORTING INTENTIONALLY

DEFAULTS-AND-PATHS.SH

```
# Required env var check -- fail fast with a clear message, not a mysterious downstream error
: "${API_TOKEN:?API_TOKEN must be set (see docs/config.md)}"

# Default config path -- respects XDG, falls back sanely
readonly CONFIG="${APP_CONFIG:-${XDG_CONFIG_HOME:-$HOME/.config}/app/config.toml}"

# Derive basename/dirname without spawning external processes
full="/srv/releases/2026.07/build.tar.gz"
base="${full##*/}"     # build.tar.gz
dir="${full%/*}"       # /srv/releases/2026.07

# Safe path construction -- never string-concat with a bare slash assumption
join_path() { local base=$1 leaf=$2; printf '%s/%s\n' "${base%/}" "${leaf#/}"; }

export APP_ENV="${APP_ENV:-production}"   # exported ON PURPOSE: children need it
LOCAL_CACHE_TTL=60                        # NOT exported: internal to this script only
```

§ 06

ARRAYS & MAPS

INDEXED | ASSOCIATIVE | ARGV BUILDING

Arrays are the antidote to string-built commands. Every time a command line gets assembled by concatenating strings with spaces, an array should have been used instead.

06.1 INDEXED ARRAYS: BUILD, APPEND, ITERATE

INDEXED-ARRAYS.SH

```
declare -a items=("first" "second item" "third")
items+=("fourth")                       # append
items+=("fifth" "sixth")                # append multiple

for it in "${items[@]}"; do             # GOOD: preserves spaces, one element per iteration
  printf '> %s\n' "$it"
done

echo "${#items[@]}"                     # length: 6
echo "${items[-1]}"                     # last element (Bash 4.3+)
unset 'items[1]'                        # delete an index -- leaves a HOLE, array is now sparse
```

SPARSE ARRAYS AFTER UNSET

After `unset 'items[1]'`, indices are no longer contiguous. `${#items[@]}` still reports the correct count, but iterating by numeric index (`for ((i=0;i<n;i++))`) will skip the hole silently. Prefer `"${items[@]}"` iteration, or re-pack with `items=("${items[@]}")`.

06.2 ASSOCIATIVE ARRAYS (BASH 4+)

ASSOC-ARRAYS.SH

```
declare -A handler=(
  [.tar.gz]="extract_targz"
  [.zip]="extract_zip"
  [.log]="rotate_log"
)

ext=".zip"
if [[ -v handler[$ext] ]]; then         # KEY EXISTENCE check (Bash 4.3+), distinct from non-empty check
  "${handler[$ext]}"
else
  die "no handler for extension: $ext"
fi

for ext in "${!handler[@]}"; do         # ! prefix -> iterate KEYS, not values
  printf '%s -> %s\n' "$ext" "${handler[$ext]}"
done
```

`[[ -V ARR[KEY] ]]`

Distinguishes "key exists with empty value" from "key never set." `[[ -n "${arr[$key]}" ]]` only tells you the value is non-empty.

06.3 ARRAYS AS COMMAND ARGUMENTS

ARGV-ARRAYS.SH

```
# Building an rg command safely -- flags and pattern never collide, spaces survive
rg_args=(--hidden --line-number)
[[ -n "${glob:-}" ]] && rg_args+=(--glob "$glob")
rg_args+=(-- "$pattern" "$search_dir")
rg "${rg_args[@]}"

# Building a curl command safely -- headers and body assembled without string concatenation
curl_args=(--fail-with-body --show-error --silent --request POST)
curl_args+=(--header "Content-Type: application/json")
curl_args+=(--header "Authorization: Bearer ${TOKEN}")
curl_args+=(--data "$json_body" -- "$url")
curl "${curl_args[@]}"
```

WHY ARRAYS BEAT STRING-BUILT COMMANDS

`cmd="rg $flags $pattern"; $cmd` re-splits on whitespace and re-globs -- any space or glob character inside `$pattern` corrupts the invocation. Arrays preserve exact argument boundaries with no re-parsing step.

06.4 DEDUPLICATION WITH ASSOCIATIVE ARRAYS

DEDUPE.SH

```
declare -A seen=()
deduped=()
for x in "${input[@]}"; do
  [[ -v seen[$x] ]] && continue
  seen[$x]=1
  deduped+=("$x")
done
```

06.5 PASSING ARRAYS TO FUNCTIONS

ARRAY-TO-FUNCTION.SH

```
process() {
  local -n items_ref=$1     # nameref -- receives the array BY REFERENCE, no copy
  local it
  for it in "${items_ref[@]}"; do
    printf 'processing: %s\n' "$it"
  done
}
my_files=("a.log" "b.log")
process my_files            # pass the NAME, not "${my_files[@]}"
```

WARNING

Passing `"${arr[@]}"` directly into a function flattens it into positional parameters — the function can iterate `"$@"` but loses the array identity (no re-indexing back into an array without a second declaration). Use namerefs when the callee needs the array itself, not just its elements.

§ 07

ARGUMENT PARSING

MANUAL PARSER | SUBCOMMANDS | -- SENTINEL

Manual `while`/`case` parsing beats `getopts` for anything with long options, and beats a third-party arg-parsing library for anything that doesn't need one.

07.1 FULL PARSER: --FILE, --OUTPUT, --DRY-RUN, --VERBOSE

FULL-PARSER.SH

```
usage() {
  cat <<EOF
usage: ${PROG} --file FILE [--output DIR] [--dry-run] [--verbose] [--] [extra...]
EOF
}

file="" output="." dry_run=0 verbose=0
while (( $# )); do
  case "$1" in
    --file)      file=${2:?--file requires a value}; shift 2 ;;
    --file=*)    file=${1#*=}; shift ;;
    --output)    output=${2:?--output requires a value}; shift 2 ;;
    --output=*)  output=${1#*=}; shift ;;
    --dry-run)   dry_run=1; shift ;;
    --verbose|-v) verbose=1; shift ;;
    -h|--help)   usage; exit 0 ;;
    --)          shift; break ;;                # explicit end-of-options sentinel
    -*)          die "unknown option: $1" ;;     # refuse silently-ambiguous input
    *)           break ;;                        # first non-option: stop, rest are positionals
  esac
done
[[ -n "$file" ]] || { usage; die "--file is required"; }
positionals=("$@")
```

--FLAG=VALUE AND --FLAG VALUE

Supporting both `--file=x` and `--file x` costs one extra case arm per flag and matches what users expect from GNU-style tools.

07.2 SUBCOMMAND PARSER

SUBCOMMAND-PARSER.SH

```
(( $# )) || { usage; exit 2; }
cmd=$1; shift
case "$cmd" in
  build)  cmd_build "$@" ;;
  deploy) cmd_deploy "$@" ;;
  status) cmd_status "$@" ;;
  -h|--help) usage; exit 0 ;;
  *) die "unknown command: '$cmd' (see --help)" ;;
esac
```

07.3 REPEATED FLAGS & REFUSING AMBIGUITY

REPEATED-FLAGS.SH

```
includes=()
while (( $# )); do
  case "$1" in
    --include) includes+=("${2:?}"); shift 2 ;;   # repeatable: -I a -I b -I c
    --*=*)     die "ambiguous: use '--flag value', not '$1'" ;;  # explicit refusal, not a guess
    *) break ;;
  esac
done
```

SILENT AMBIGUITY IS WORSE THAN A CRASH

A parser that guesses at malformed input (e.g. treating an unrecognized `--flag=x=y` as flag `x=y`) fails hours later in a way that's hard to trace. Refuse and exit non-zero on anything you didn't explicitly design for.

07.4 GETOPTS: LIMITED BUT USEFUL

GETOPTS.SH

```
while getopts ":o:vh" opt; do
  case "$opt" in
    o) output=$OPTARG ;;
    v) verbose=1 ;;
    h) usage; exit 0 ;;
    :) die "option -$OPTARG requires an argument" ;;
    \?) die "unknown option: -$OPTARG" ;;
  esac
done
shift $((OPTIND - 1))
```

> **GETOPTS CEILING**
>
> `getopts` only handles single-character flags natively (no `--long-form` without manual case-matching on `-l` plus a lookup), doesn't support repeated multi-value flags cleanly, and resets `OPTIND` semantics differently across shells. Fine for small internal tools; switch to manual parsing the moment you need long options.

07.5 WHEN TO SWITCH TO PYTHON INSTEAD

Once argument validation needs nested subcommands with per-subcommand help text, type coercion, or config-file merging with CLI precedence, `argparse` or `click` will be less code and fewer bugs than the Bash equivalent. A rough signal: if the parser itself would exceed ~80 lines, reconsider the language, not just the parsing strategy.

§ 08

FUNCTIONS & RETURN BEHAVIOR

STDOUT=DATA | STDERR=LOGS | LIBRARY FILES

The single rule that keeps functions composable: **stdout carries data, stderr carries logs, the return code carries status**. Violate this and every caller has to guess which output stream is safe to capture.

08.1 STATUS VS. OUTPUT – NEVER MIX THEM

STDOUT-DISCIPLINE.SH

```
# BAD: mixes a log line into the data stream -- caller's capture is now corrupted
get_version() {
  echo "fetching version..."      # pollutes stdout
  echo "2.4.1"
}
v=$(get_version)                  # v="fetching version...\n2.4.1"  -- wrong

# GOOD: logs to stderr, data to stdout, status via return
get_version() {
  printf 'fetching version...\n' >&2
  printf '2.4.1\n'
  return 0
}
v=$(get_version)                  # v="2.4.1" -- clean
```

08.2 CORE LIBRARY FUNCTIONS

LIB-CORE.SH

```
die()  { printf 'fatal: %s\n' "$*" >&2; exit 1; }
log()  { printf '[%s] %s\n' "$1" "$2" >&2; }        # log LEVEL message

require_cmd() {
  local c
  for c in "$@"; do
    command -v "$c" >/dev/null 2>&1 || die "missing required command: $c"
  done
}

confirm() {                                          # confirm "prompt" -- returns 0/1, no output leak
  local reply
  read -r -p "${1:-Continue?} [y/N] " reply
  [[ $reply =~ ^[Yy]$ ]]
}

join_by() {                                          # join_by , a b c -> "a,b,c"
  local sep=$1; shift
  local out=$1; shift
  printf '%s' "$out"
  printf "%s" "${@/#/$sep}"
}

contains() {                                         # contains needle "${haystack[@]}"
  local needle=$1; shift
  local x
  for x in "$@"; do [[ $x == "$needle" ]] && return 0; done
  return 1
}
```

08.3 RETRY — CAPTURING FAILURE WITHOUT LOSING STATUS

RETRY.SH

```
retry() {                       # retry <max_attempts> <sleep_seconds> -- cmd...
  local max=$1 delay=$2; shift 2
  local attempt=1 rc=0
  until "$@"; do
    rc=$?
    (( attempt >= max )) && return "$rc"
    log WARN "attempt $attempt/$max failed (rc=$rc), retrying in ${delay}s: $*"
    sleep "$delay"
    (( attempt++ ))
  done
  return 0
}
retry 3 2 curl -fsS "$url" -o "$out"
```

08.4 WITH_TEMPDIR & CLEANUP – TRAPS INSIDE FUNCTIONS

WITH-TEMPDIR.SH

```
with_tempdir() {                # with_tempdir <callback-fn> [args...]
  local dir cb=$1; shift
  dir=$(mktemp -d) || die "mktemp -d failed"
  trap 'rm -rf -- "$dir"' RETURN              # RETURN trap: fires when THIS function returns
  "$cb" "$dir" "$@"
}
process_in_scratch() { local dir=$1; printf 'work happens in %s\n' "$dir" > "$dir/marker"; }
with_tempdir process_in_scratch
```

TRAP ... RETURN

A `RETURN` trap is local to the function it's set in and fires on every return from that function (normal or via an early `return`) — cleaner than a hand-rolled cleanup call before every exit point.

08.5 PASS-BY-NAME WITH NAMEREFS

PASS-BY-NAME.SH

```
collect_into() {                # collect_into <out-array-name> <cmd...>
  local -n out=$1; shift
  local line
  while IFS= read -r line; do out+=("$line"); done < <("$@")
}
declare -a results=()
collect_into results find . -maxdepth 1 -name '*.sh'
```

08.6 LIBRARY-STYLE FILES & SOURCE SAFETY

LIB-STRINGS.SH

```
# lib/strings.sh -- meant to be sourced, never executed directly
[[ "${BASH_SOURCE[0]}" != "$0" ]] || {
  printf '%s: this file is a library, source it instead\n' "${BASH_SOURCE[0]##*/}" >&2
  exit 1
}

trim() { local s=$1; s="${s#"${s%%[![:space:]]*}"}"; s="${s%"${s##*[![:space:]]}"}"; printf '%s' "$s"; }
```

GUARD PATTERN EXPLAINED

`"${BASH_SOURCE[0]}" != "$0"` is true only when the file is being `source`d (sourced files keep the caller's `$0`); it's false when executed directly, at which point the guard exits with a clear message instead of silently running library code as a script.

§ 09

FILESYSTEM-SAFE SCRIPTING

NUL DELIMITERS | ATOMIC WRITES | SAFE DELETION

Filenames may contain spaces, newlines, glob characters, or leading dashes. Every pattern below exists because a real filename eventually breaks the naive version.

09.1 NUL-DELIMITED LOOPS — THE ONLY SAFE FIND LOOP

NULL-DELIMITED-LOOP.SH

```
# BAD: for f in $(find ...) -- word-splits on spaces AND newlines, breaks on any of them
for f in $(find . -name '*.log'); do rm "$f"; done

# GOOD: NUL-delimited, survives spaces, newlines, everything but NUL itself (impossible in filenames)
while IFS= read -r -d '' f; do
  rm -- "$f"
done < <(find . -name '*.log' -print0)
```

WHY NUL, NOT NEWLINE

Newline is a legal character in a Unix filename. NUL (`\0`) is the only byte that cannot appear in a filename, which is exactly why `-print0` / `-d ''` / `-0` exist as a matched set.

09.2 XARGS -0 AND THE -- SENTINEL

XARGS-NULL.SH

```
find . -name '*.tmp' -print0 | xargs -0 -r rm --      # -r: don't run if input is empty
# -- stops rm from interpreting a filename that happens to start with '-' as a flag
touch -- '-rf'; rm -- '-rf'
```

09.3 CHECKING FILE TYPES & SYMLINK HANDLING

FILE-TYPE-CHECKS.SH

```
for f in "$dir"/*; do
  if [[ -L "$f" ]]; then                # test -L BEFORE -f/-d: a symlink can also pass -f
    log INFO "skipping symlink: $f"
    continue
  elif [[ -f "$f" ]]; then
    process_file "$f"
  elif [[ -d "$f" ]]; then
    process_dir "$f"
  fi
done
```

09.4 ATOMIC WRITES — TEMP FILE + MV

ATOMIC-WRITE.SH

```
atomic_write() {                        # atomic_write <target> <content-on-stdin>
  local target=$1 tmp
  tmp=$(mktemp "${target}.XXXXXX") || die "mktemp failed"
  cat > "$tmp" || { rm -f "$tmp"; die "write failed"; }
  chmod --reference="$target" "$tmp" 2>/dev/null || true    # GNU-specific; see &sect;12 for a portable fallback
  mv -- "$tmp" "$target"                # mv within the same filesystem is atomic (POSIX rename())
}
printf 'new config\n' | atomic_write /etc/app/config.toml
```

MV ACROSS FILESYSTEMS ISN'T ATOMIC

The atomicity guarantee only holds when the temp file and target share a filesystem — `mktemp "${target}.XXXXXX"` (same directory as the target) guarantees this. `mktemp` alone often lands in `/tmp`, a different mount.

09.5 SAFE BACKUP BEFORE DESTRUCTIVE EDIT

BACKUP-BEFORE-EDIT.SH

```
backup_and_edit() {
  local f=$1
  [[ -f "$f" ]] || die "not a file: $f"
  local bak="${f}.bak.$(date -u +%Y%m%dT%H%M%SZ)"
  cp -p -- "$f" "$bak" || die "backup failed: $f"
  # ... perform edit on $f, e.g. via atomic_write ...
  log INFO "backed up $f -> $bak"
}
```

09.6 DRY-RUN DELETE REQUIRING EXPLICIT CONFIRMATION

SAFE-RM.SH

```
safe_rm() {                             # safe_rm [--force] path...
  local force=0
  [[ ${1:-} == --force ]] && { force=1; shift; }
  local p
  for p in "$@"; do
    guard_path "$p" || die "refusing to operate on suspicious path: $p"
    if (( force )); then
      rm -rf -- "$p"
    else
      printf 'would remove: %s\n' "$p"
    fi
  done
}
```

09.7 REFUSING TO OPERATE ON /, EMPTY PATHS, OR $HOME

GUARD-PATH.SH

```
guard_path() {
  local p=$1
  [[ -n "$p" ]] || return 1                    # empty string -> refuse
  [[ "$p" != "/" ]] || return 1                # root -> refuse
  [[ "$p" != "$HOME" ]] || return 1            # exact $HOME -> refuse
  local resolved
  resolved=$(cd -- "$(dirname -- "$p")" 2>/dev/null && pwd -P)/$(basename -- "$p") || return 1
  [[ "$resolved" != "/" && "$resolved" != "$HOME" ]] || return 1
  return 0
}
```

FOOTGUN

A variable that expands to empty inside `rm -rf "$target_dir"/` collapses to `rm -rf /`-adjacent behavior on the current directory tree. `guard_path`-style checks belong in front of every destructive call, not just the ones that feel risky.

09.8 LOCK FILES FOR SINGLE-INSTANCE SCRIPTS

LOCK-FILE.SH

```
readonly LOCK="/var/run/${PROG}.lock"
exec 9> "$LOCK"
flock -n 9 || die "another instance is already running"
# ... critical section ...
# lock released automatically when fd 9 closes at process exit
```

FLOCK IS UTIL-LINUX (LINUX)

`flock` ships with util-linux and is not present on macOS by default. Portable fallback: a `mkdir`-based lock, since `mkdir` is atomic on every POSIX filesystem.

§ 10

PIPELINES, PROCESS SUBSTITUTION & SUBSHELLS

VARIABLE SCOPE | <( ) | STATUS CAPTURE

Every stage of a pipeline runs in its own subshell. Variables set inside a pipeline stage do not survive past the pipeline — this single fact explains most "my counter stayed zero" bugs.

10.1 THE CLASSIC WHILE READ PIPELINE BUG

PIPELINE-SUBSHELL-BUG.SH

```
count=0
grep -c "" file | while read -r n; do count=$n; done
# WRONG in some contexts too, but illustrating the trap:
find . -name '*.log' | while read -r f; do (( count++ )); done
echo "$count"                          # prints 0 -- the while ran in a subshell, count++ never escaped it
```

PROCESS-SUBSTITUTION-FIX.SH FIXED

```
count=0
while IFS= read -r -d '' f; do
  (( count++ ))
done < <(find . -name '*.log' -print0)     # process substitution: NOT a subshell for the while itself
echo "$count"                           # correct count -- the loop ran in the CURRENT shell
```

WHY < <(...) FIXES IT

Process substitution replaces `<(...)` with a path to a pipe / fd; the `while` loop reads from that path in the CURRENT shell, not inside a pipe stage. No subshell boundary, no variable loss.

10.2 CAPTURING OUTPUT AND STATUS SEPARATELY

CAPTURE-STATUS.SH

```
out=$(some_cmd 2>err.log)
rc=$?                                  # capture immediately -- the NEXT command overwrites $?
if (( rc != 0 )); then
  die "some_cmd failed (rc=$rc): $(cat err.log)"
fi
```

10.3 SPLITTING STDOUT/STDERR WHILE PRESERVING BOTH

SPLIT-STREAMS.SH

```
# tee into a log file while still letting stdout flow to the terminal / caller
build_output=$(make 2>&1 | tee -a build.log)

# Separate stdout and stderr into different files while checking pipeline status
{ cmd 2>&3 | tee stdout.log; } 3>&1 1>&2 | tee stderr.log
```

THE 3>&1 1>&2 IDIOM

Duplicates fd 1 to fd 3, then swaps fds 1 and 2, so the second `tee` in the pipeline receives what was originally stderr. Dense but standard — worth memorizing once rather than re-deriving.

10.4 GROUPING { ...; } VS. SUBSHELL (...)

GROUPING-VS-SUBSHELL.SH

```
{ echo "a"; echo "b"; } > combined.log     # group: runs in CURRENT shell, no fork
( cd /tmp && rm -f scratch.txt )            # subshell: cd only affects the subshell, caller's cwd
unchanged
echo "back in $PWD"                         # unaffected by the subshell's cd
```

10.5 PIPEFAIL IN PRACTICE

PIPEFAIL-PRACTICE.SH

```
set -o pipefail
curl -fsS "$url" | jq -r '.data[]'
echo "exit status: $?"     # non-zero if EITHER curl OR jq failed, not just jq (the last stage)
```

10.6 CONSUMING JQ OUTPUT SAFELY

CONSUME-JQ-SAFELY.SH

```
while IFS= read -r line; do
  process "$line"
done < <(jq -r '.items[] | @base64' response.json)   # base64 each item -> one safe line per element, no
                                                      # ambiguity from embedded newlines/tabs in the
JSON values
```

NEWLINES INSIDE JSON VALUES

Piping `jq -r` output straight into `while read` breaks if any field contains an embedded newline. The `@base64` + per-item `jq -r` decode pattern (full example in §16) sidesteps this entirely.

§ 11

TEXT PROCESSING PATTERNS

AWK VS SED VS JQ | REGEX | WHEN TO STOP

Pick the tool by what it's structurally good at, not by habit: parameter expansion for single-value string edits, `awk` for field-oriented data, `sed` for line-oriented substitution, `jq` for JSON, Python the moment state or real data structures enter the picture.

11.1 DECISION TABLE

TOOL	USE WHEN	STOP WHEN
Parameter expansion	Single variable, one substitution/trim.	You need a loop or a regex group.
awk	Column/field-oriented text, running totals.	You need real data structures or JSON.
sed	Line-oriented substitution, simple filters.	The expression needs a lookahead or captures beyond \N.
perl	sed/awk outgrew themselves but a full script is overkill.	Team doesn't maintain Perl — readability tax isn't worth it.
jq	Any JSON in or out.	The transform needs external state or I/O.
Python	Multiple data sources, real error handling, testability.	— (ceiling tool)

11.2 REGEX MATCHING WITH [[=~]] AND BASH_REMATCH

BASH-REGEX.SH

```
line="2026-07-02 ERROR service=api rc=503"
if [[ $line =~ ^([0-9-]+)\ ([A-Z]+)\ service=([a-z]+)\ rc=([0-9]+)$ ]]; then
  date=${BASH_REMATCH[1]}; level=${BASH_REMATCH[2]}
  svc=${BASH_REMATCH[3]};  rc=${BASH_REMATCH[4]}
  printf '%s [%s] %s rc=%s\n' "$date" "$level" "$svc" "$rc"
fi
```

QUOTE THE PATTERN VARIABLE, NOT THE REGEX

If the regex itself is stored in a variable, do NOT quote it in the `[[ =~ ]]` test — quoting forces literal string matching instead of regex interpretation in Bash 3.2+. `[[ $s =~ $pattern ]]`, not `[[ $s =~ "$pattern" ]]`.

11.3 EXTRACT FIELDS WITH AWK

AWK-FIELDS.SH

```
awk -F',' '{ printf "%s: %s\n", $1, $3 }' data.csv
awk '/ERROR/ { count++ } END { print count+0 }' app.log      # count without a Bash loop
```

11.4 PARSING A SIMPLE .ENV / KEY=VALUE CONFIG

PARSE-ENV.SH

```
declare -A cfg=()
while IFS='=' read -r key val; do
  [[ $key =~ ^[[:space:]]*(#.*)?$ ]] && continue   # skip blanks and comments
  key=${key%%[[:space:]]*}
  val=${val%\"}; val=${val#\"}                     # strip one layer of surrounding double quotes, if
present
  cfg[$key]=$val
done < .env
```

11.5 EDITING FILES WITHOUT INCOMPATIBLE SED -I

SED-I-PORTABILITY.SH

```
# sed -i BEHAVIOR DIFFERS: GNU requires no space before the suffix arg (or none at all);
# BSD/macOS REQUIRES an explicit (possibly empty) suffix argument.
sed -i 's/foo/bar/' file        # GNU: in place, no backup
sed -i '' 's/foo/bar/' file     # BSD/macOS: '' means no backup, and IS a required separate argument

# Portable pattern: write to a temp file, then atomic mv (see &sect;09) -- works identically everywhere
sed 's/foo/bar/' file > file.tmp && mv -- file.tmp file
```

SED -I FOOTGUN

Scripts that run `sed -i 's/x/y/' f` on Linux and are then run unmodified on macOS will either error ("missing extension") or, worse, create a file literally named `'s/x/y/'`. The temp-file + `mv` pattern sidesteps the incompatibility entirely.

11.6 JQ -R SAFELY

JQ-SAFELY.SH

```
jq -r '.results[] | select(.status=="active") | [.id, .name] | @tsv' data.json |
  while IFS=$'\t' read -r id name; do
    printf 'active: %s (%s)\n' "$name" "$id"
  done
```

-R AVOIDS DOUBLE-QUOTED STRINGS

Without `-r`, `jq` emits JSON-quoted strings (with surrounding `"`) into what should be plain text — a frequent source of silently-wrong downstream comparisons.

11.7 CSV, JSON, AND YAML FOOTGUNS

- **CSV**: fields can contain embedded commas, quotes, and newlines. A naive `awk -F,` or `IFS=,` split silently mis-parses quoted fields — use a real CSV-aware tool (`csvkit`, Python's `csv`) once fields can be quoted.
- **JSON**: hand-built JSON via string concatenation breaks the moment a value contains a quote, backslash, or Unicode. Always build with `jq -n` (§16), never `printf`.
- **YAML**: Bash has no YAML-aware tool in the base system. Parsing YAML with `grep` / `sed` works until the first multi-line string, anchor, or nested list — reach for `yq` (jq-like, YAML-aware) or Python at the first sign of structure.

§ 12

EXTERNAL COMMANDS & DEPENDENCY MANAGEMENT

GNU VS BSD | VERSION GUARDS | COMMAND -V

Scripts that run only on the author's machine aren't portable; scripts that detect their environment and fail with a clear message are.

12.1 REQUIRE_CMD AND COMMAND -V, NOT WHICH

REQUIRE-CMD.SH

```
require_cmd() {
  local c
  for c in "$@"; do
    command -v "$c" >/dev/null 2>&1 || die "missing required command: $c (install it and retry)"
  done
}
require_cmd jq curl git
```

COMMAND -V OVER WHICH

`which` is not POSIX-specified, behaves inconsistently across systems, and some implementations don't report shell builtins/functions at all. `command -v` is a shell builtin, POSIX-specified, and correctly reports functions, aliases, builtins, and external commands alike.

12.2 DETECTING GNU VS. BSD COREUTILS

DETECT-GNU-BSD.SH

```
is_gnu_sed() { sed --version >/dev/null 2>&1; }   # GNU sed supports --version; BSD sed errors on it

if is_gnu_sed; then
  SED=(sed -i)
else
  SED=(sed -i '')      # BSD/macOS requires the explicit (empty) backup-suffix argument
fi
"${SED[@]}" 's/foo/bar/' file
```

12.3 GNU-ONLY TOOLS AND THEIR ALTERNATIVES

GNU (LINUX DEFAULT)	BSD/MACOS DEFAULT	HOMEBREW FALLBACK
`date -d`	`date -j -f` (different syntax entirely)	`gdate` (coreutils)
`sed -i 's/x/y/'`	`sed -i '' 's/x/y/'`	`gsed`
`readlink -f`	not available (macOS default)	`greadlink -f`
`find -print0 / -regextype`	`-print0` ok; `-regextype` differs	`gfind`
`stat -c '%s'`	`stat -f '%z'`	`gstat`

> **HOMEBREW'S G-PREFIX CONVENTION**
>
> `brew install coreutils` installs GNU tools prefixed with `g` (`gsed`, `gdate`, ...) specifically to avoid shadowing the BSD originals macOS system tooling expects.

12.4 PORTABLE-ISH MKTEMP

PORTABLE-MKTEMP.SH

```
# GNU mktemp and BSD mktemp both support this minimal form; avoid --suffix/-p (GNU-only flags)
tmp=$(mktemp "${TMPDIR:-/tmp}/app.XXXXXX") || die "mktemp failed"
```

12.5 MACOS-SAFE DATE HANDLING

> **DATE REQUIRES VERIFICATION**
>
> GNU `date -d '1 day ago' +%F` has no BSD equivalent flag-for-flag; BSD/macOS date uses `-v-1d` instead. There is no single incantation that works unmodified on both — branch on `date --version` success/failure, or depend on GNU coreutils being installed and require it explicitly. **Requires verification on your specific target OS before relying on either form in production.**

PORTABLE-DATE.SH

```
if date --version >/dev/null 2>&1; then
  yesterday=$(date -d 'yesterday' +%F)          # GNU
else
  yesterday=$(date -v-1d +%F)                   # BSD/macOS
fi
```

12.6 VERSION GUARDS

VERSION-GUARD.SH

```
require_bash_version() {
  local need=$1
  (( BASH_VERSINFO[0] >= need )) || die "requires Bash >= $need (found ${BASH_VERSION})"
}
require_bash_version 4       # associative arrays, &sect;06, need Bash 4+
```

12.7 AVOIDING ALIASES/FUNCTIONS WITH COMMAND

COMMAND-BYPASS.SH

```
ls() { echo "wrapped"; }         # a user's shell rc might define this
command ls -la /tmp              # bypasses functions AND aliases, calls the real external binary
```

§ 13

LOGGING, VERBOSITY & UX

LOG LEVELS | TTY-AWARE COLOR | --JSON

A script's UX is judged by how it fails, not how it succeeds. Leveled logs, a usage string that's actually useful, and errors that name the offending value are most of the work.

13.1 LEVELED LOGGING

LEVELED-LOGGING.SH

```
: "${LOG_LEVEL:=INFO}"
declare -A LOG_LEVELS=([DEBUG]=0 [INFO]=1 [WARN]=2 [ERROR]=3)

_log() {
  local level=$1; shift
  (( LOG_LEVELS[$level] >= LOG_LEVELS[$LOG_LEVEL] )) || return 0
  printf '%s [%-5s] %s\n' "$(date -u +%FT%TZ)" "$level" "$*" >&2
}
log_debug() { _log DEBUG "$@"; }
log_info()  { _log INFO  "$@"; }
log_warn()  { _log WARN  "$@"; }
log_error() { _log ERROR "$@"; }
```

13.2 TTY-AWARE COLOR WITH NO_COLOR

TTY-COLOR.SH

```
is_tty() { [[ -t 2 ]]; }            # check stderr specifically -- that's where logs go

if is_tty && [[ -z "${NO_COLOR:-}" ]]; then
  C_RED=$'\033[31m'; C_YEL=$'\033[33m'; C_RST=$'\033[0m'
else
  C_RED=""; C_YEL=""; C_RST=""
fi
log_error() { printf '%s[ERROR]%s %s\n' "$C_RED" "$C_RST" "$*" >&2; }
```

RESPECT NO_COLOR

The `NO_COLOR` convention (no-color.org) is checked by many tools users already rely on. Honoring it costs three lines and avoids fighting the user's terminal setup or log-aggregation pipeline.

13.3 QUIET / VERBOSE / DEBUG MODES

VERBOSITY-MODES.SH

```
VERBOSITY=1     # 0=quiet 1=normal 2=verbose 3=debug
while (( $# )); do
  case "$1" in
    -q|--quiet)   VERBOSITY=0; shift ;;
    -v|--verbose) (( VERBOSITY++ )); shift ;;    # repeatable: -vv -> debug
    *) break ;;
  esac
done
(( VERBOSITY >= 3 )) && set -x
```

13.4 CLEAN USAGE TEXT

USAGE-TEXT.SH

```
usage() {
  cat <<EOF
usage: ${PROG} [options] <input>

options:
  -o, --output DIR   write results to DIR (default: .)
  -v, --verbose       increase verbosity (repeatable)
  -q, --quiet         suppress non-error output
      --json          emit machine-readable JSON to stdout
  -h, --help          show this help and exit

examples:
  ${PROG} --output ./out data.csv
  ${PROG} -vv --json data.csv | jq '.summary'
EOF
}
```

13.5 --JSON OUTPUT MODE

JSON-OUTPUT-MODE.SH

```
emit_result() {
  local name=$1 status=$2 count=$3
  if (( JSON_MODE )); then
    jq -n --arg n "$name" --arg s "$status" --argjson c "$count" \
      '{name:$n, status:$s, count:$c}'
  else
    printf '%-20s %-8s %d items\n' "$name" "$status" "$count"
  fi
}
```

HUMAN OUTPUT ON STDOUT BREAKS --JSON CONSUMERS

If any log line ever leaks to stdout in `--json` mode, downstream `jq` parsing breaks on that line. The stdout-for-data discipline from §08 is what makes `--json` mode trustworthy — enforce it especially carefully here.

13.6 MEANINGFUL ERRORS — NAME THE OFFENDING VALUE

MEANINGFUL-ERRORS.SH

```
# BAD
[[ -f "$file" ]] || die "file not found"

# GOOD -- names the exact value and, ideally, what was expected
[[ -f "$file" ]] || die "file not found: '$file' (resolved from --input=$raw_input)"
```

§ 14

TRAPS, CLEANUP & SIGNALS

EXIT/INT/TERM/ERR | EXIT STATUS | CHILD REAPING

One `trap EXIT` handler, set once near the top of the script, beats cleanup calls scattered before every `exit` and `return`.

14.1 THE STANDARD TRAP SET

STANDARD-TRAPS.SH

```
TMPDIR_LOCAL=""
CHILD_PIDS=()

cleanup() {
  local rc=$?                                  # MUST be the first statement -- capture before anything
else runs
  trap - EXIT INT TERM                         # disarm to avoid re-entrant cleanup on a second signal
  (( ${#CHILD_PIDS[@]} )) && kill "${CHILD_PIDS[@]}" 2>/dev/null
  [[ -n "$TMPDIR_LOCAL" && -d "$TMPDIR_LOCAL" ]] && rm -rf -- "$TMPDIR_LOCAL"
  exit "$rc"
}
trap cleanup EXIT
trap 'cleanup; exit 130' INT      # 130 = 128+SIGINT, conventional exit code for Ctrl-C
trap 'cleanup; exit 143' TERM     # 143 = 128+SIGTERM
```

RC=$? MUST BE THE FIRST LINE

Any command before `local rc=$?` inside the trap handler — even an innocuous-looking `[[ -n "$x" ]]` — overwrites `$?` before you capture it, silently turning every failure into exit code 0 downstream.

14.2 ERR TRAP WITH LINE NUMBER CONTEXT

ERR-TRAP-CONTEXT.SH

```
trap 'on_err "$LINENO" "$BASH_COMMAND"' ERR
on_err() {
  local line=$1 cmd=$2
  log_error "line $line: command failed: $cmd"
}
```

14.3 CLEANUP ORDER MATTERS

Reverse the order things were acquired: release locks before removing the directory that contains the lock file; kill children before removing the temp dir they may still be writing into; flush and close log file descriptors last, so cleanup itself stays logged.

14.4 KILLING CHILD PROCESSES ON EXIT

CHILD-CLEANUP.SH

```
start_worker() {
  "$@" &
  CHILD_PIDS+=($!)
}
start_worker sleep 300
start_worker tail -f app.log
# cleanup() above already kills everything in CHILD_PIDS on EXIT/INT/TERM
```

PROCESS GROUPS FOR DEEP TREES

If a child spawns its own children, killing just the direct PID leaves grandchildren orphaned. `kill -- -"$pgid"` (negative PID) signals the whole process group instead — pair with `set -m` or `setsid` to control grouping.

14.5 PRESERVING THE ORIGINAL EXIT CODE THROUGH CLEANUP

PRESERVE-EXIT-CODE.SH

```
main() {
  run_the_actual_work
}
main "$@"
rc=$?
cleanup_side_effects        # this call's own exit status must NOT leak into $?
exit "$rc"
```

14.6 TRAPS INSIDE SUBSHELLS

SUBSHELLS GET THEIR OWN TRAP TABLE

A trap set inside `( ... )` applies only within that subshell and is reset to default on entry — it does not inherit the parent's EXIT trap, and setting one there does not affect the parent. If a subshell needs cleanup, it needs its own trap.

SUBSHELL-TRAP.SH

```
(
  trap 'rm -f "$scratch"' EXIT     # local to this subshell only
  scratch=$(mktemp)
  do_work_in_subshell "$scratch"
)   # scratch is cleaned up here; the parent shell's own EXIT trap is untouched
```

14.7 TRAP STACKING & DISABLING TRAPS

TRAP-STACKING.SH

```
# Bash has no native trap stack -- compose manually if a library function needs to add
# its own EXIT behavior without clobbering the caller's:
_old_exit_trap=$(trap -p EXIT)
trap 'eval "${_old_exit_trap#trap -- }"; my_extra_cleanup' EXIT

# Disable a trap entirely when its handler must not run for a controlled, expected exit:
trap - EXIT
exit 0
```

§ 15

CONCURRENCY & BATCHING

BACKGROUND JOBS | WAIT -N | FAILURE COLLECTION

Bash concurrency is coarse — processes, not threads — but `wait -n` plus a job-count guard covers most batch-processing needs without reaching for GNU parallel.

15.1 RUNNING N JOBS AT ONCE

N-JOBS-AT-ONCE.SH

```
max_jobs=4
pids=()
for f in *.log; do
  process_one "$f" &
  pids+=($!)
  while (( $(jobs -rp | wc -l) >= max_jobs )); do
    wait -n           # Bash 4.3+: block until ANY one background job finishes
  done
done
wait "${pids[@]}"       # drain the rest
```

WAIT -N AVAILABILITY

Requires Bash 4.3+. On older Bash (notably stock macOS Bash 3.2), fall back to a semaphore-file or fixed-size-batch approach — see §20 for the full portability matrix.

15.2 COLLECTING FAILURES WITHOUT LOSING ANY

COLLECT-FAILURES.SH

```
declare -A job_pid_to_input=()
pids=() failures=()

for f in "${inputs[@]}"; do
  process_one "$f" & job_pid_to_input[$!]=$f
  pids+=($!)
done

for pid in "${pids[@]}"; do
  wait "$pid" || failures+=("${job_pid_to_input[$pid]}")
done

if (( ${#failures[@]} )); then
  log_error "failed: ${failures[*]}"
  exit 1
fi
```

WAIT WITHOUT AN EXPLICIT PID LOSES THE MAPPING

A single `wait` (no args) blocks for all jobs but returns only the LAST job's exit status. Waiting on each PID individually, as above, is what lets you attribute a specific failure to a specific input.

15.3 PER-JOB LOGS, NO SHARED-FILE RACES

PER-JOB-LOGS.SH

```
run_job() {
  local input=$1 logdir=$2
  local logfile="$logdir/$(basename -- "$input").log"
  process_one "$input" > "$logfile" 2>&1     # each job owns its own output file -- no interleaving, no
race
}
mkdir -p "$LOGDIR"
for f in "${inputs[@]}"; do run_job "$f" "$LOGDIR" & done
wait
```

FOOTGUN

Multiple background jobs appending to the SAME file (`cmd >> shared.log`) is not atomic per-write on every filesystem/kernel combination for writes larger than `PIPE_BUF` — output can interleave mid-line. Give every job its own file and concatenate afterward if a single log is needed.

15.4 XARGS -P PARALLELISM

XARGS-PARALLEL.SH

```
find . -name '*.png' -print0 | xargs -0 -P 4 -I{} convert {} -resize 50% {}.thumb.png
```

GNU PARALLEL FOR ANYTHING MORE THAN THIS

The moment you need per-job retries, a progress bar, load-aware throttling, or result aggregation, `parallel` (GNU parallel, separate install) does it in one invocation instead of hand-rolled job control. Not installed by default on macOS or most minimal Linux images — treat as an explicit dependency, gated by §12's `require_cmd` .

15.5 ATOMIC OUTPUTS FROM PARALLEL JOBS

Combine the atomic-write pattern from §09 with per-job temp files: each job writes to its own `mktemp` path, and only the final aggregation step — running single-threaded after `wait` — merges results into the shared output. This avoids every form of write-write race by construction rather than by locking.

§ 16

NETWORKING & API SCRIPTING

CURL DISCIPLINE | JQ -N PAYLOADS | PAGINATION

Every production `curl` call needs an explicit failure mode, an explicit timeout, and an explicit place tokens come from. None of these are curl defaults.

16.1 BASELINE CURL FLAGS

CURL-BASELINE.SH

```
curl \
  --fail-with-body \      # non-2xx -> non-zero exit, but body is still printed/captured (curl 7.76+)
  --show-error \          # print error message even with --silent
  --silent \              # no progress meter
  --location \            # follow redirects
  --connect-timeout 5 \   # cap connection phase
  --max-time 30 \         # cap the ENTIRE request, including transfer
  --retry 3 --retry-connrefused --retry-delay 2 \
  "$url"
```

--FAIL-WITH-BODY VS. -F

Plain `-f` / `--fail` discards the response body on error, which throws away exactly the JSON error message you need for logging. `--fail-with-body` (curl ≥7.76) keeps both the non-zero exit AND the body.

16.2 CAPTURING HTTP STATUS AND BODY SEPARATELY

STATUS-AND-BODY.SH

```
response=$(curl -sS -o /tmp/resp.$$ -w '%{http_code}' "$url"); rc=$?
body=$(cat /tmp/resp.$$); rm -f /tmp/resp.$$
if (( rc != 0 )); then die "curl transport error (rc=$rc)"; fi
if (( response >= 400 )); then die "HTTP $response: $body"; fi
```

16.3 AUTH VIA ENVIRONMENT, NEVER HARDCODED

AUTH-ENV.SH

```
: "${API_TOKEN:?set API_TOKEN (e.g. from your secrets manager) before running this script}"
curl -sS -H "Authorization: Bearer ${API_TOKEN}" "$url"

# NEVER: curl -H "Authorization: Bearer sk-live-abc123..." -- hardcoded secrets end up in
# shell history, process listings (ps aux shows argv!), and version control.
```

SECRETS IN ARGV ARE VISIBLE VIA PS

A token passed as a literal command-line argument is visible to every other user on the box via `ps aux` for the life of the process. Environment variables are visible only via `/proc/PID/environ` to the same user/root — still not perfectly private, but meaningfully better. Full treatment in §17.

16.4 JSON PAYLOADS WITH JQ -N

JQ-N-PAYLOAD.SH

```
payload=$(jq -n \
  --arg name "$user_name" \
  --arg email "$user_email" \
  --argjson active true \
  '{name: $name, email: $email, active: $active}')

curl -sS --fail-with-body -X POST "$url" \
  -H 'Content-Type: application/json' \
  -d "$payload"
```

NEVER PRINTF JSON BY HAND

`printf '{\"name\":\"%s\"}' "$name"` breaks the instant `$name` contains a quote, backslash, or control character. `jq -n --arg` handles all JSON escaping correctly by construction.

16.5 PAGINATION PATTERN

PAGINATION.SH

```
page=1 all_results="[]"
while :; do
  resp=$(curl -sS --fail-with-body "$url?page=$page&per_page=100")
  batch=$(jq -c '.items' <<< "$resp")
  all_results=$(jq -c --argjson a "$all_results" --argjson b "$batch" '$a + $b' <<< null)
  next=$(jq -r '.has_more' <<< "$resp")
  [[ "$next" == "true" ]] || break
  (( page++ ))
done
jq '.' <<< "$all_results"
```

16.6 RATE-LIMIT HANDLING

RATE-LIMIT.SH

```
status=$(curl -sS -o /tmp/resp.$$ -w '%{http_code}' "$url")
if [[ "$status" == "429" ]]; then
  retry_after=$(curl -sSI "$url" | awk -F': ' 'tolower($1)=="retry-after"{print $2}' | tr -d '\r')
  sleep "${retry_after:-5}"
fi
```

16.7 TLS VERIFICATION — DO NOT DISABLE CASUALLY

WARNING

`curl -k` / `--insecure` disables certificate validation entirely, silently accepting any certificate including an attacker's. Valid uses are limited to a known-and-controlled internal CA that isn't in the system trust store — fix that with `--cacert /path/to/ca.pem` instead of disabling verification.

§ 17

SECURITY HARDENING

INJECTION DEFENSE | PATH HYGIENE | SHELLCHECK GATES

Most Bash security incidents are quoting bugs and trust-boundary confusion, not exotic exploits. The defenses below are the same handful of patterns from earlier sections, collected as a checklist.

17.1 INJECTION DEFENSE CHECKLIST

- No `eval` on anything derived from user input, network responses, or file contents you don't fully control (§04.10).
- No unquoted variable expansions outside arithmetic context (§04.1).
- No command lines built by string concatenation — use arrays for argv (§06.3).
- No secrets or user input interpolated directly into `awk` / `sed` program text (§04.9).
- No `ssh host "$user_controlled_string"` without a second layer of quoting for the remote shell (§04.9).

17.2 INPUT VALIDATOR FOR PATHS

VALIDATE-PATH.SH

```
validate_path() {
  local p=$1
  [[ -n "$p" ]] || { log_error "empty path"; return 1; }
  [[ "$p" != *$'\n'* ]] || { log_error "path contains newline: refusing"; return 1; }
  case "$p" in
    /*) : ;;                              # absolute: fine
    ../*|*/../*|*/..) log_error "path traversal attempt: $p"; return 1 ;;
    *) : ;;                               # relative, no traversal segment: fine
  esac
  return 0
}
```

17.3 PATH HYGIENE & COMMAND SPOOFING

PATH-HYGIENE.SH

```
# A hostile or accidental entry in PATH (e.g. "." first) lets a same-named file shadow
# a real system binary. Set PATH explicitly at the top of security-sensitive scripts:
export PATH="/usr/bin:/bin:/usr/sbin:/sbin"

# Prefer absolute paths for anything privilege-sensitive:
/usr/bin/sudo -u appuser /opt/app/bin/migrate

# command bypasses shell functions/aliases that might shadow a builtin or external tool (&sect;12.7)
command rm -rf -- "$scratch_dir"
```

A LEADING . IN PATH

If `PATH` contains `.` (current directory) — common in old or misconfigured shell rc files — running `ls` in a directory containing a malicious file named `ls` executes that file instead of the real one. Never add `.` to `PATH`.

17.4 SAFE SECRET LOADING FROM ENVIRONMENT

SECRET-LOADING.SH

```
# GOOD: read from a secrets manager / env, never store in the script or in argv
: "${DB_PASSWORD:?DB_PASSWORD must be set via your secrets manager}"

# GOOD: read from a restricted-permission file, never echoed
read -r DB_PASSWORD < /run/secrets/db_password

# BAD: never do this
# DB_PASSWORD="hunter2"                          # hardcoded, ends up in git history
# mysql -p"$DB_PASSWORD"                          # visible in `ps aux`
```

17.5 UMASK AND SECURE TEMP FILES

UMASK-SECURE-TEMPDIR.SH

```
umask 077                           # new files/dirs default to owner-only (0600 / 0700)
secure_dir=$(mktemp -d) || die "mktemp -d failed"    # mktemp already creates with 0700
trap 'rm -rf -- "$secure_dir"' EXIT
```

UMASK IS PROCESS-WIDE

Set `umask 077` as early as possible, before any file is created, and be aware it affects every subsequent file creation in the script — including unrelated output files the user may expect to be group-readable.

17.6 ABSOLUTE PATHS & DENY-BY-DEFAULT CHECKS

DENY-BY-DEFAULT.SH

```
readonly ALLOWED_ROOTS=(/srv/app/data /srv/app/uploads)
path_is_allowed() {
  local target; target=$(cd -- "$(dirname -- "$1")" 2>/dev/null && pwd -P) || return 1
  local root
  for root in "${ALLOWED_ROOTS[@]}"; do
    [[ "$target" == "$root" || "$target" == "$root"/* ]] && return 0
  done
  return 1                          # deny by default -- not in the allow-list, refuse
}
```

17.7 AVOIDING CURL | BASH AND UNSAFE SUDO

> **WARNING**
>
> `curl https://example.com/install.sh | bash` executes whatever the server returns AT THE MOMENT OF THE REQUEST, with no integrity check — a compromised server, MITM, or simple server-side bug runs arbitrary code with your privileges. Download, inspect (or checksum-verify), then execute as two separate steps. For `sudo` : never `sudo bash -c "$user_string"` — the quoting failure modes from §04 now run as root.

17.8 SHELLCHECK AS A GATE, NOT A SUGGESTION

SHELLCHECK-GATE.SH

```
# CI step -- fail the build on any ShellCheck finding at warning level or above
shellcheck -x -S warning ./bin/*.sh ./lib/*.sh || exit 1
```

> **-X FOLLOWS SOURCE**
>
> `-x` tells ShellCheck to follow `source` d files instead of warning that it can't see them — required for any script that uses the library pattern from §08.6.

§ 18

DEBUGGING & INSPECTION

BASH -X | PS4 TRACING | MINIMAL REPROS

A well-chosen `PS4` turns `bash -x` output from noise into a stack trace. Set it up once and reuse it across every script.

18.1 SYNTAX CHECK WITHOUT EXECUTING

SYNTAX-CHECK.SH

```
bash -n script.sh          # parse only -- catches syntax errors, never runs a single command
```

18.2 PRETTY PS4 WITH LINE NUMBERS & FUNCTION NAMES

PS4-TRACING.SH

```
export PS4='+ ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}() '
bash -x ./script.sh 2>&1 | less

# tracing only a section, without a full -x run:
set -x
suspect_function "$@"
set +x
```

DEFAULT PS4 IS NEARLY USELESS

The default `PS4` is just `+ ` — every traced line looks identical with no source location. The custom form above is worth putting in a shared snippet.

18.3 TRACING TO A FILE, NOT THE TERMINAL

XTRACEFD.SH

```
exec {BASH_XTRACEFD}> /tmp/trace.$$.log   # Bash 4.1+: dedicated fd for xtrace output
set -x
# ... noisy section ...
set +x
exec {BASH_XTRACEFD}>&-
```

BASH_XTRACEFD

Redirects `set -x` output away from stderr entirely, so trace noise doesn't interleave with real error output — useful when a script's stderr is itself being parsed by a caller.

18.4 INSPECTING VARIABLES SAFELY

DECLARE-P.SH

```
declare -p my_array my_assoc_array my_var     # shows type, attributes, and exact value/quoting
typeset -p SOME_VAR 2>/dev/null || echo "SOME_VAR is unset"
```

DECLARE -P OVER ECHO FOR DEBUGGING

`declare -p` shows the REAL stored value including embedded whitespace and quoting attributes — `echo "$var"` can hide a trailing space or an empty-vs-unset distinction that's exactly the bug you're chasing.

18.5 SHELLCHECK & SHFMT

SHELLCHECK-SHFMT.SH

```
shellcheck -x script.sh          # static analysis: quoting, unreachable code, common footguns
shfmt -d -i 2 -ci script.sh       # -d: show a diff instead of rewriting; -i 2: 2-space indent
```

18.6 MINIMAL REPRO SCRIPTS

When a bug only reproduces in a 400-line script, strip it down before debugging further: comment out `set -e` temporarily, replace external calls with fixed sample data, and delete everything not on the path to the failing line. A 10-line repro that fails 100% of the time is worth more than a 400-line script that fails 30% of the time under conditions you don't yet understand.

18.7 A DEBUG WRAPPER FUNCTION

DEBUG-WRAPPER.SH

```
dbg() {                              # dbg <command...> -- runs a command, echoing it and its rc
  (( ${DEBUG:-0} )) || { "$@"; return; }
  printf 'DEBUG> %s\n' "$*" >&2
  "$@"; local rc=$?
  printf 'DEBUG< rc=%s\n' "$rc" >&2
  return "$rc"
}
dbg curl -sS "$url"
```

§ 19

TESTING BASH SCRIPTS

SMOKE TESTS | BATS-CORE | PATH MOCKING

Bash scripts are testable like any other program once the logic is split into functions and the side-effecting parts (network, filesystem writes) are isolated behind thin wrappers.

19.1 MINIMAL TEST HARNESS WITHOUT A FRAMEWORK

MINIMAL-HARNESS.SH

```
#!/usr/bin/env bash
set -Eeuo pipefail
pass=0 fail=0

assert_eq() {
  local desc=$1 expected=$2 actual=$3
  if [[ "$expected" == "$actual" ]]; then
    printf 'ok      %s\n' "$desc"; (( pass++ ))
  else
    printf 'FAIL    %s\n        expected: %q\n        actual:   %q\n' "$desc" "$expected" "$actual"
    (( fail++ ))
  fi
}

source ./lib/strings.sh
assert_eq "trim strips leading space" "x" "$(trim '  x')"
assert_eq "trim strips trailing space" "x" "$(trim 'x  ')"

printf '\n%d passed, %d failed\n' "$pass" "$fail"
(( fail == 0 ))
```

19.2 GOLDEN OUTPUT TESTS

GOLDEN-TESTS.SH

```
run_and_diff() {
  local name=$1
  ./bin/report.sh test/fixtures/"$name".input > /tmp/"$name".actual
  diff -u test/golden/"$name".expected /tmp/"$name".actual
}
run_and_diff basic_report
```

19.3 MOCKING COMMANDS WITH PATH

PATH-MOCKING.SH

```
setup_mocks() {
  MOCK_DIR=$(mktemp -d)
  cat > "$MOCK_DIR/curl" <<'EOF'
#!/usr/bin/env bash
echo '{"status":"ok"}'
EOF
  chmod +x "$MOCK_DIR/curl"
  PATH="$MOCK_DIR:$PATH"          # mocked curl shadows the real one, only for this test process
}
```

WHY PATH MOCKING BEATS FUNCTION STUBS

Overriding `curl` as a shell function only intercepts calls from within the SAME shell — it doesn't affect subshells, background jobs, or anything invoked via `env`. A PATH-shadowed executable intercepts every invocation regardless of subshell depth.

19.4 TESTING DESTRUCTIVE CODE VIA DRY-RUN

TEST-DRY-RUN.SH

```
test_safe_rm_dry_run() {
  local tmp; tmp=$(mktemp)
  local out; out=$(safe_rm "$tmp")           # no --force -> dry-run path only
  assert_eq "dry-run does not delete" "1" "$([[ -f "$tmp" ]] && echo 1 || echo 0)"
  assert_eq "dry-run reports the path" "would remove: $tmp" "$out"
  rm -f "$tmp"
}
```

19.5 BATS-CORE EXAMPLE

STRINGS.BATS

```
# test/strings.bats
setup() { source "${BATS_TEST_DIRNAME}/../lib/strings.sh"; }

@test "trim strips leading and trailing space" {
  run trim "  hello  "
  [ "$status" -eq 0 ]
  [ "$output" = "hello" ]
}

@test "contains finds an element" {
  run contains "b" a b c
  [ "$status" -eq 0 ]
}
```

BATS IS NOT PREINSTALLED

`bats-core` is a separate install (`brew install bats-core` / `npm i -g bats` / distro package). Gate its use in CI behind an explicit install step, same as any other dependency (§12).

19.6 CI INTEGRATION: SHELLCHECK + SHFMT + BATS

CI-YAML.TXT

```
# .github/workflows/ci.yml (excerpt)
- name: ShellCheck
  run: shellcheck -x -S warning bin/*.sh lib/*.sh
- name: shfmt
  run: shfmt -d -i 2 -ci .
- name: Bats
  run: bats test/
```

§ 20

PORTABILITY MATRIX

BASH 3.2-5.X | ZSH | POSIX SH

macOS ships Bash 3.2 (last GPLv2 release, frozen since 2007) as `/bin/bash` for licensing reasons. Every feature below marked 4+/5+ silently fails or behaves differently there — this is the single most common source of "works on my Linux box, breaks for the Mac user" reports.

20.1 FEATURE AVAILABILITY BY SHELL/VERSION

FEATURE	BASH 3.2 (MACOS DEFAULT)	BASH 4.X	BASH 5.X	ZSH	POSIX SH
Associative arrays	no	yes	yes	yes	no
`${var,,}` / `${var^^}`	no	yes	yes	different syntax	no
`wait -n`	no	4.3+	yes	no	no
`mapfile` / `readarray`	no	yes	yes	via `zsh/mapfile`	no
`declare -n` (nameref)	no	4.3+	yes	no	no
`${arr[-1]}` negative index	no	4.3+	yes	1-indexed, different syntax	no
Process substitution `<(...)`	yes	yes	yes	yes	no
`[[ ]]` conditional	yes	yes	yes	yes	no
Arrays (indexed)	yes	yes	yes	yes	no
`$EPOCHSECONDS`	no	no	5.0+	yes	no

20.2 GNU VS. BSD/MACOS USERLAND

COMMAND	GNU (LINUX)	BSD (MACOS DEFAULT)
`sed -i`	no arg needed for no backup	requires explicit (possibly empty) suffix arg
`stat`	`stat -c '%s'`	`stat -f '%z'`
`date`	`date -d 'STRING'`	`date -j -f FMT STRING` or `-v` offsets
`readlink -f`	built in	not present by default (no canonicalize flag)
`find -regextype`	supported	not supported (BSD find regex differs entirely)
`cp --reflink`	supported (CoW filesystems)	not supported

20.3 READLINK -F: THE SILENT MACOS GAP

PORTABLE-READLINK.SH

```
# GNU: works out of the box
real=$(readlink -f "$path")

# Portable fallback (POSIX-ish, no GNU dependency):
real_path() {
  local p=$1
  ( cd -- "$(dirname -- "$p")" && printf '%s/%s\n' "$(pwd -P)" "$(basename -- "$p")" )
}
real=$(real_path "$path")
```

READLINK -F IS NOT PORTABLE TO MACOS BY DEFAULT

Absent unless GNU coreutils is installed via Homebrew (as `greadlink -f`). The `real_path()` fallback above works everywhere but doesn't resolve symlinks in intermediate path segments the way `readlink -f` does — verify it meets your actual requirement before relying on it for symlink-security-sensitive code.

20.4 MAPFILE/READARRAY CAVEATS

MAPFILE-FALLBACK.SH

```
mapfile -t lines < file            # Bash 4+ only; -t strips trailing newlines
# On Bash 3.2 (macOS default bash) or when targeting POSIX sh, fall back to a while-read loop:
lines=()
while IFS= read -r line || [[ -n "$line" ]]; do
# the || [[ -n ]] catches a final line with no trailing newline
  lines+=("$line")
done < file
```

§ 21

ADVANCED COMBO PATTERNS

FIELD-TESTED | 11 SPELLS | PROBLEM → WHY → FOOTGUN

Full patterns, not fragments. Each one has shipped in production form; adapt the specifics, keep the shape.

01 / GIT + RIPGREP Safe Changed-Files Audit

Problem. Scan only files changed in the current branch for a risky pattern, without breaking on filenames with spaces.

CHANGED-FILES-AUDIT.SH

```
base=$(git merge-base HEAD origin/main) || die "no merge-base with origin/main"
git diff --name-only -z "$base"...HEAD -- '*.sh' |
  xargs -0 -r rg --line-number --with-filename 'eval\s' -- ||
    { rc=$?; (( rc == 1 )) && exit 0; exit "$rc"; }   # rg exit 1 = no matches, not an error
```

Why it's shaped this way. git diff --name-only -z NUL-delimits output (spaces/unicode-safe); xargs -0 hands each null-delimited name to rg without a shell re-split. rg's exit code 1 means 'no matches' — treated as success, not failure.

Footguns. Forgetting -z on the git side re-introduces word-splitting on the xargs side even though xargs itself is null-safe. rg exit codes: 0=match found, 1=no match, 2=real error — collapsing all non-zero to failure breaks the common case of a clean scan.

02 / FILESYSTEM Recursive File Processor: NUL-Delimited + Dry-Run

Problem. Recursively touch every matching file under a root, defaulting to a safe preview.

RECURSIVE-PROCESSOR.SH

```
process_tree() {
  local root=$1 dry_run=${2:-1}
  [[ -d "$root" ]] || die "not a directory: $root"
  local f
  while IFS= read -r -d '' f; do
    if (( dry_run )); then
      printf 'would process: %s\n' "$f"
    else
      process_one "$f"
    fi
  done < <(find "$root" -type f -name '*.tmp' -print0)
}
process_tree "$1" "${DRY_RUN:-1}"
```

Why it's shaped this way. dry_run defaults to 1 (on) — a destructive tree-walk requires an explicit opt-out, not opt-in, matching the §09 dry-run convention. -print0/-r -d '' keeps every filename intact regardless of spaces or glob chars.

Footguns. A default of dry_run=0 here would be the classic footgun: the first real run of a new script silently becomes the destructive one. Keep safe-by-default even when it costs an extra flag at the callsite.

03 / NETWORKING API Fetcher: Retries, Timeouts, JSON Parsing

Problem. Fetch from a flaky API, retrying transient failures without retrying real 4xx errors.

API-FETCHER.SH

```
fetch_json() {
  local url=$1 attempt=1 max=4 resp status
  while (( attempt <= max )); do
    resp=$(curl -sS -o /tmp/resp.$$ -w '%{http_code}' --connect-timeout 5 --max-time 20 "$url")
    status=$resp
    if (( status >= 200 && status < 300 )); then
      cat /tmp/resp.$$; rm -f /tmp/resp.$$; return 0
    elif (( status >= 400 && status < 500 )); then
      rm -f /tmp/resp.$$; die "client error $status, not retrying: $url"
    fi
    rm -f /tmp/resp.$$
    sleep $(( attempt * 2 ))
    (( attempt++ ))
  done
  die "exhausted $max attempts: $url"
}
fetch_json "$url" | jq -r '.data[]'
```

Why it's shaped this way. 4xx is treated as a permanent failure (bad request, bad auth) and fails fast; 5xx and transport errors retry with linear backoff. Splitting these two failure classes avoids hammering a server that's telling you the request itself is wrong.

Footguns. Every 429 (rate limit) technically falls in the retryable range here but ignores Retry-After (§16.6) — fine for low-volume scripts, insufficient against a strict rate limiter.

04 / CLEANUP Temporary Workspace With Preserved Exit Status

Problem. Run a multi-step job in an isolated scratch directory that's always cleaned up, without losing the real exit code.

TEMPDIR-WORKSPACE.SH

```
main() {
  WORKDIR=$(mktemp -d) || die "mktemp -d failed"
  trap 'rm -rf -- "$WORKDIR"' EXIT
  step_one "$WORKDIR"
  step_two "$WORKDIR"
  step_three "$WORKDIR"
}
main "$@"
rc=$?
exit "$rc"
```

Why it's shaped this way. The EXIT trap fires regardless of which step fails, on any exit path (normal, error, or signal-driven via the INT/TERM traps from §14). Capturing rc explicitly after main keeps the caller's real status intact instead of whatever the trap's own last command returned.

Footguns. If any step function itself sets an EXIT trap, it silently overwrites this one (see §14.7 for stacking). Keep trap-setting centralized in main() for scripts with this shape.

05 / CONCURRENCY Parallel Batch Processor With Failure Collection

Problem. Process a large file list with bounded parallelism and a clear pass/fail summary at the end.

PARALLEL-BATCH.SH

```
process_batch() {
  local max=${MAX_JOBS:-4}
  declare -A pid_to_file=()
  local failures=() f pid rc
  for f in "$@"; do
    process_one "$f" & pid_to_file[$!]=$f
    while (( $(jobs -rp | wc -l) >= max )); do wait -n; done
  done
  for pid in "${!pid_to_file[@]}"; do
    wait "$pid" || failures+=("${pid_to_file[$pid]}")
  done
  (( ${#failures[@]} == 0 )) || { log_error "failed: ${failures[*]}"; return 1; }
}
process_batch "${files[@]}"
```

Why it's shaped this way. Bounding with wait -n instead of processing in fixed-size chunks keeps all worker slots busy — a chunk boundary otherwise stalls waiting for the slowest item in each chunk before starting the next.

Footguns. wait -n needs Bash 4.3+ (§20); on older Bash, replace with a semaphore-file count or fixed-size xargs -P batching instead.

06 / CONFIG Config-Driven Script Using Associative Arrays

Problem. Drive script behavior entirely from a declarative map instead of a chain of if/elif.

CONFIG-DRIVEN.SH

```
declare -A ACTIONS=(
  [deploy]="do_deploy"
  [rollback]="do_rollback"
  [status]="do_status"
)
declare -A REQUIRES_CONFIRM=([deploy]=1 [rollback]=1 [status]=0)

run_action() {
  local action=$1
  [[ -v ACTIONS[$action] ]] || die "unknown action: $action"
  if [[ "${REQUIRES_CONFIRM[$action]}" == 1 ]]; then
    confirm "Really run '$action'?" || die "aborted"
  fi
  "${ACTIONS[$action]}"
}
```

Why it's shaped this way. Adding a new action means adding two map entries, not another case arm scattered through the control flow — the map IS the documentation of what the script can do.

Footguns. [[-v ACTIONS[$action]]] checks key existence; [[-n ${ACTIONS[$action]:-}]] would also (wrongly) reject a deliberately empty-string action value. Use -v when 'defined' and 'non-empty' need to differ.

07 / FILESYSTEM Safe Backup-and-Edit Workflow

Problem. Edit a live config file with a timestamped backup and an atomic swap, never a half-written file on disk.

SAFE-BACKUP-EDIT.SH

```
safe_edit() {
  local target=$1 editor_fn=$2
  [[ -f "$target" ]] || die "not a file: $target"
  local bak="${target}.bak.$(date -u +%Y%m%dT%H%M%SZ)"
  cp -p -- "$target" "$bak" || die "backup failed"
  local tmp; tmp=$(mktemp "${target}.XXXXXX") || die "mktemp failed"
  "$editor_fn" < "$target" > "$tmp" || { rm -f "$tmp"; die "edit failed, original untouched"; }
  mv -- "$tmp" "$target"          # atomic same-filesystem replace
  log_info "edited $target (backup: $bak)"
}
```

Why it's shaped this way. Every failure path (backup fails, edit fails) leaves the ORIGINAL file untouched — the target is only ever replaced by the atomic mv at the very end, once a fully-written replacement exists.

Footguns. cp -p preserves mode/timestamps but not ACLs/xattrs on every platform — verify on target OS if those matter. The backup is never cleaned up automatically; add retention/rotation for long-running services.

08 / AUDIT Shell Startup File Auditor

Problem. Flag risky patterns across a user's shell startup files before they cause a support ticket.

RC-AUDITOR.SH

```
audit_rc_files() {
  local f patterns=('curl.*\|.*bash' 'eval .*\$' 'PATH=.*:\.[:$]')
  for f in ~/.bashrc ~/.bash_profile ~/.zshrc ~/.profile; do
    [[ -f "$f" ]] || continue
    local p
    for p in "${patterns[@]}"; do
      grep -nE "$p" -- "$f" 2>/dev/null | while IFS=: read -r line _; do
        printf '%s:%s: matches risky pattern: %s\n' "$f" "$line" "$p"
      done
    done
  done
}
```

Why it's shaped this way. Patterns target real incident classes: curl-pipe-bash left in an rc file, eval on an expanding variable, and a leading '.' in PATH (§17.3) — all silent until the exact moment they aren't.

Footguns. grep -E patterns here are illustrative, not exhaustive — this is a smoke-test audit, not a substitute for reading the files. False negatives are expected for anything obfuscated.

09 / DEPENDENCIES Dependency / Version Checker

Problem. Fail fast and clearly when a required tool is missing or too old, instead of failing obscurely mid-script.

DEP-CHECKER.SH

```
check_deps() {
  require_cmd jq curl git
  local jq_major; jq_major=$(jq --version | grep -oE '[0-9]+' | head -1)
  (( jq_major >= 1 )) || die "jq >=1.6 required, found: $(jq --version)"
  require_bash_version 4
}
check_deps
```

Why it's shaped this way. Every dependency check happens in one place, at the top of main, before any real work starts — a missing tool fails in one line instead of forty lines into a partially-completed run.

Footguns. Version-string parsing via grep -oE is fragile against format changes between tool versions; pin to a known-good range and re-verify the parsing when bumping a minimum version.

10 / CI CI Guard That Fails on Forbidden Patterns

Problem. Block a merge if new code introduces a banned pattern (eval, curl|bash, hardcoded secrets).

CI-FORBIDDEN-PATTERNS.SH

```
readonly FORBIDDEN=(
  'eval "\$'
  'curl[^|]*\| *bash'
  '(password|token|secret)[[:space:]]*='
)
violations=0
while IFS= read -r -d '' f; do
  for pat in "${FORBIDDEN[@]}"; do
    if grep -nE "$pat" -- "$f" >/dev/null 2>&1; then
      grep -nE "$pat" -- "$f" | sed "s|^|$f:|"
      (( violations++ ))
    fi
  done
done < <(git diff --name-only -z --diff-filter=ACM origin/main -- '*.sh')
(( violations == 0 )) || die "$violations forbidden pattern(s) found"
```

Why it's shaped this way. Runs only against ADDED/CHANGED files (--diff-filter=ACM) against origin/main, not the whole repo — keeps the check fast and focused on what the PR actually introduces.

Footguns. Regex-based secret detection has a real false-negative rate; treat this as a fast pre-filter ahead of a dedicated secret scanner (gitleaks, trufflehog), not a replacement for one.

11 / PORTABILITY macOS-Safe Script That Detects GNU/BSD Differences

Problem. One script, correct behavior on both a Linux CI runner and a developer's Mac, without two code paths everywhere.

USERLAND-DETECT.SH

```
detect_userland() {
  if sed --version >/dev/null 2>&1; then USERLAND=gnu; else USERLAND=bsd; fi
}
sed_inplace() {                       # sed_inplace 'expr' file -- one call site, works on both
  local expr=$1 file=$2
  if [[ "$USERLAND" == gnu ]]; then
    sed -i "$expr" "$file"
  else
    sed -i '' "$expr" "$file"
  fi
}
detect_userland
sed_inplace 's/foo/bar/' config.txt
```

Why it's shaped this way. Detection happens once, at startup, into a single USERLAND variable; every callsite that cares (here, just sed_inplace) branches on that cached value instead of re-probing per call.

Footguns. This pattern only covers the tools you've explicitly wrapped. Any direct GNU-flag call elsewhere in the script (readlink -f, date -d, stat -c) still needs its own portability treatment per §12/§20.

§ 22

APPENDIX

REFERENCE TABLES | QUICK LOOKUP | SHELLCHECK CODES

Compact reference tables. No prose — this section is for scanning, not reading top to bottom.

A.1 TEST OPERATORS ([[ ]] AND [ ])

OPERATOR	MEANING
`-e`	path exists (any type)
`-f` / `-d`	regular file / directory
`-L`	symlink (test before -f/-d)
`-r` / `-w` / `-x`	readable / writable / executable
`-s`	exists and size > 0
`-z` / `-n`	string is empty / non-empty
`-v`	variable (or array key) is set
`==` / `!=`	string equality (glob-aware inside `[[ ]]`)
`=~`	regex match, only inside `[[ ]]`
`-eq -ne -lt -le -gt -ge`	numeric comparison
`-nt` / `-ot`	file A newer/older than file B

A.2 PARAMETER EXPANSION QUICK REFERENCE

FORM	RESULT
`${var:-x}`	x if unset/empty, var unchanged
`${var:=x}`	x if unset/empty, AND assigns
`${var:?msg}`	error+exit if unset/empty
`${var:+x}`	x if var IS set, else empty
`${#var}`	length of var
`${var:off:len}`	substring
`${var#pat}` / `##pat`	strip shortest / longest matching prefix
`${var%pat}` / `%%pat`	strip shortest / longest matching suffix
`${var/pat/repl}`	replace first match
`${var//pat/repl}`	replace all matches
`${var^}` / `${var^^}`	uppercase first char / all (Bash 4+)
`${var,}` / `${var,,}`	lowercase first char / all (Bash 4+)
`${!var}`	indirect expansion

A.3 ARRAY OPERATIONS

FORM	RESULT
`arr=(a b c)`	indexed array literal
`declare -A m=([k]=v)`	associative array literal
`arr+=(x)`	append
`"${arr[@]}"`	all elements, individually quoted
`"${arr[*]}"`	all elements joined by IFS[0] into one word
`"${!arr[@]}"`	all indices/keys
`${#arr[@]}`	element count
`unset 'arr[i]'`	delete one element (leaves a hole)
`[[ -v arr[k] ]]`	key exists (Bash 4.3+)

A.4 REDIRECTION FORMS

FORM	EFFECT
`> file`	stdout to file, truncate
`>> file`	stdout to file, append
`2>file`	stderr to file
`>file 2>&1`	both streams to file (order matters)
`&>file`	Bash shorthand for both streams to file
`<<EOF`	heredoc
`<<'EOF'`	heredoc, no expansion inside
`<<< "$str"`	herestring
`<(cmd)` / `>(cmd)`	process substitution, read/write side
`exec 3<file`	open a file on a numbered fd for later use

A.5 TRAP PATTERNS

FORM	USE
`trap cleanup EXIT`	run on every exit path
`trap 'h $LINENO' ERR`	run on any command failure (needs `set -E` to reach into functions)
`trap '' INT`	ignore SIGINT entirely
`trap - EXIT`	disarm/reset a trap to default
`trap -p EXIT`	print the currently registered handler
`trap cb RETURN`	run when the current function returns (function-local)

A.6 SET OPTIONS

OPTION	EFFECT
`-e`	exit on unhandled non-zero simple command
`-u`	error on unset variable expansion
`-o pipefail`	pipeline status = rightmost non-zero stage
`-E`	ERR trap inherited by functions/subshells
`-x`	print each command before executing (trace)
`-n`	parse only, don't execute
`-v`	print each line of input as read
`-C`	refuse to clobber existing files with `>`

A.7 COMMON EXIT CODES

CODE	MEANING
0	success
1	general error (convention, not enforced)
2	misuse of shell builtin / bad usage (convention)
126	command found but not executable
127	command not found
128+N	terminated by signal N (130 = SIGINT, 143 = SIGTERM)

A.8 QUOTING RULES — THE SHORT VERSION

- Double quotes: variable and command substitution happen; word splitting and globbing do not.
- Single quotes: nothing expands, ever — the only way to get a truly literal string.
- `$'...'`: ANSI-C quoting — backslash escapes (`\n`, `\t`) are interpreted, no variable expansion.
- Unquoted: word splitting AND globbing both apply — the default state to avoid.
- Inside `[[ ]]`: word splitting and globbing never apply, even unquoted (but quote anyway for consistency and pattern-matching clarity).

A.9 SAFE PIPELINE RULES

- `set -o pipefail` before any pipeline whose failure must propagate.
- Never rely on a variable set inside a `| while read` pipeline surviving past it — use process substitution instead (§10.1).
- Capture `$?` immediately after command substitution — the next command overwrites it.
- Prefer `find -print0 | xargs -0` or process substitution over `for x in $(cmd)`.

A.10 COMMON BASH FOOTGUNS – ONE-LINE REMINDERS

FOOTGUN	FIX
Unquoted `$var`	Quote it: `"$var"`
`for f in $(find ...)`	`while read -r -d '' f; do ... done < <(find ... -print0)`
`rm -rf $dir/*`, empty dir	`rm -rf -- "${dir:?}"/*`
`cmd1 | cmd2` swallows cmd1's failure	`set -o pipefail`
`count++` inside `| while` doesn't persist	process substitution, not a pipe
`eval` on any user-influenced string	arrays / namerefs instead
`sed -i` across GNU/BSD	temp file + atomic `mv`
Trap handler doesn't capture `$?` first	`local rc=$?` as the very first line
Secrets as CLI args	environment variables / secrets manager
`"${arr[*]}"` where `"${arr[@]}"` was meant	use `@` unless you specifically want one joined string

A.11 SHELLCHECK WARNING EXAMPLES & MEANINGS

CODE	MEANING
SC2086	double quote to prevent globbing/word splitting
SC2046	quote this to prevent word splitting of a command substitution
SC2034	variable appears unused
SC2154	variable referenced but never assigned in this file
SC2181	check exit code directly, not via `$?` after the fact
SC2064	use single quotes in `trap` unless you want expansion NOW, not at trap time
SC2068	double quote array expansions to avoid re-splitting elements
SC2206/SC2207	prefer `mapfile` / `read -a` over unquoted array assignment from command substitution
SC2164	`cd` without `|| exit` can silently continue in the wrong directory